

Università di Firenze
Scuola di Scienze Matematiche Fisiche e Naturali
Laurea Magistrale in Data Science, Calcolo Scientifico & Intelligenza
Artificiale

**Apprendimento per Rinforzo
Multi-Agente basato su Reti
Neurali a Grafo per il Controllo
Semaforico**

Relatore

Daniele Castellana

Candidato

Alessio Minati

Anno Accademico 2025/2026

Sommario

Il controllo semaforico su una rete di intersezioni è un problema di coordinamento distribuito: la fase scelta a un incrocio influenza, con pochi secondi di ritardo, il traffico che raggiunge gli incroci vicini. Le soluzioni classiche, a tempi fissi o reattive sulla sola pressione locale (MaxPressure), non tengono conto di questa dipendenza spaziale in modo esplicito. MetaSTGAT (Wang et al., 2022) affronta il problema con un agente di reinforcement learning per intersezione, coordinato da una *graph attention network* i cui pesi sono generati dinamicamente da un modulo di meta-learning condizionato su caratteristiche locali della rete stradale.

Questo lavoro replica da zero l’architettura e la procedura di training di MetaSTGAT su un ambiente costruito su CITYFLOW, con reti a griglia di tre dimensioni (4×4 , 5×5 , 6×6) e traffico generato parametricamente: ogni configurazione include un’arteria a densità artificialmente maggiorata, usata per misurare l’equità del controllo tra direzioni di traffico diverse. Ogni differenza introdotta rispetto alla procedura originale, dalla ricompensa alla struttura del replay buffer, è isolata da un sistema di preset di ablation e motivata singolarmente invece di essere presentata come un insieme indistinto di miglioramenti.

I risultati sperimentali confrontano le baseline classiche (Fixed-Time, MaxPressure) con tre varianti del modello Pro che differiscono per il peso α della penalità anti-starvation ($\alpha \in \{0.08, 0.04, 0.00\}$), con lo studio di ablation completo sui sei preset previsti e con un confronto tra tre meccanismi di comunicazione spaziale a parità di resto dell’architettura: il *graph attention* meta-condizionato del modello di riferimento (Meta-GAT), una convoluzione grafica meta-condizionata (Meta-GCN) e una propagazione ondulatoria meta-condizionata ispirata a SONAR (Meta-SONAR). Rimuovere del tutto la penalità anti-starvation ($\alpha = 0$) fa collassare il max wait verso i valori di MaxPressure e allo stesso tempo peggiora il tempo di viaggio massimo rispetto alle varianti con $\alpha > 0$, escludendo un semplice compromesso tra equità e throughput. Lo studio di ablation isola un effetto analogo su scala più ampia: disattivare singolarmente i meccanismi dell’algoritmo di apprendimento (Double DQN, il replay prioritizzato, la componente ricorrente della Meta-LSTM) lascia il modello

vicino al riferimento su ogni metrica, mentre rimuovere insieme il vincolo di visibilità, il segnale di attesa nello stato e la ricompensa riscritta riporta il comportamento vicino a quello di un controllore reattivo privo di garanzie di equità.

Il confronto tra meccanismi spaziali mostra un divario netto tra la convoluzione grafica, priva di un meccanismo di compatibilità appreso tra nodi, e gli altri due: Meta-GCN resta indietro su ogni metrica, mentre Meta-GAT e Meta-SONAR restano sostanzialmente alla pari, con il modello di riferimento a un solo layer che si conferma il più equo dell'intero confronto. La rappresentazione del tempo di attesa nello stato è infine un'ultima scelta di design verificata sperimentalmente: sostituire la normalizzazione lineare, che rende indistinguibili tutte le attese oltre i 100 secondi, con una funzione $\tanh$ che continua a crescere ben oltre quella soglia migliora ogni metrica riportata, motivo per cui questa rappresentazione è adottata per l'intero parco di modelli di questo lavoro, non solo per la configurazione su cui è stata isolata.

Indice

Sommario	i
Acronimi e Notazione	ix
1 Introduzione	1
1.1 Contributo di questo lavoro	5
1.2 Struttura della tesi	6
2 Background	7
2.1 Reinforcement Learning	7
2.2 Graph Neural Network	10
2.3 Traffic Signal Control come problema	13
3 Metodologia	16
3.1 Formalizzazione del problema	16
3.2 Ambiente di simulazione	18
3.2.1 Struttura della rete e mappatura delle fasi	18
3.2.2 Gestione dell'interverde	21
3.2.3 Dataset e generazione parametrica	21
3.2.4 Assunzioni e limiti del modello di simulazione	25
3.3 Baseline classiche	27
3.4 Architettura MetaSTGAT	28
3.4.1 State Encoder duale	29
3.4.2 Modulo spazio-temporale corrente (CST)	31
3.4.3 Modulo spaziale corrente (CS)	32
3.4.4 Testa Q-value	33
3.4.5 Meta-knowledge learner	33
3.4.6 Meta-GAT	36
3.4.7 Meta-LSTM	37
3.5 Differenze rispetto al modello di riferimento	39

3.6	Meccanismi spaziali alternativi	43
3.6.1	Meta-GCN	44
3.6.2	Meta-SONAR	45
3.7	Procedura di training e differenze dal paper originale	46
4	Confronto	49
4.1	Metriche di valutazione	49
4.2	Protocollo sperimentale	51
4.3	Confronti proposti	52
4.3.1	Studio del peso α	52
4.3.2	Studio di ablazione	52
4.3.3	Studio sui meccanismi spaziali	53
4.4	Risultati	53
4.4.1	Fixed-Time, MaxPressure e Pro: il confronto di base	54
4.4.2	Studio del peso α : risultati	55
4.4.3	Studio di ablazione: risultati	58
4.4.4	Studio sui meccanismi spaziali: risultati	60
4.4.5	Tabelle dei risultati.	62

Elenco delle figure

3.1	Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.	19
3.2	Rappresentazione grafica delle 8 fasi di Tabella 3.1: le frecce indicano i movimenti abilitati da ciascuna fase.	20
3.3	Rendering nel visualizzatore di CITYFLOW delle quattro combinazioni topologia-spaziatura di Tabella 3.2.	23
3.4	Schema a blocchi dell'architettura STGAT: dallo stato dell'intersezione s_i^t , attraverso l'encoder duale (MLP) e i moduli CS e CST (GAT e LSTM a pesi condivisi), fino all'azione a_i^t scelta dal DQN. Adattato da Wang et al. (2022).	30
3.5	Principio del meta-apprendimento applicato da MetaSTGAT: le meta-feature spaziali e temporali condizionano, tramite un meta-learner dedicato, i pesi dei layer LSTM e GAT usati per elaborare lo stato corrente del traffico. Adattato da Wang et al. (2022).	34
3.6	Il Meta-knowledge Learner: le meta-feature spaziali e temporali grezze vengono elaborate rispettivamente dallo SMK-Learner e dal TMK-Learner, producendo gli embedding $SMK(i)$ e $TMK(i)$ usati da Meta-GAT e Meta-LSTM. Adattato da Wang et al. (2022).	35
3.7	Meta-GAT: il modulo CS (sinistra) genera i pesi (W_j, b_j) della propria GAT a partire da SMK , il modulo CST (destra) genera (W_k, b_k) a partire da TMK ; entrambi producono l'output z_i del rispettivo modulo. Adattato da Wang et al. (2022).	38
3.8	Meta-LSTM "srotolata" nel tempo: a ogni istante t il meta-learner genera, da $TMK(i)$, i pesi W_Φ e b_Φ della cella LSTM che produce x_i^t dall'input e_i^t e dallo stato nascosto precedente h_i^{t-1}, c_i^{t-1} . Adattato da Wang et al. (2022).	39

3.9	Schema a blocchi completo di MetaSTGAT: rispetto a STGAT (Figura 3.4), GAT e LSTM sono sostituiti dalle rispettive versioni meta-condizionate, con i pesi generati dagli embedding $SMK(i)$ e $TMK(i)$ del Meta-knowledge Learner. Adattato da Wang et al. (2022).	40
4.1	Travel time medio (fascia scura), max wait (fascia intermedia) e travel time massimo (fascia chiara) per lo sweep su α , stessi dati delle Tabelle 4.4-4.5.	57
4.2	Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) per lo studio su α , calcolata per singola configurazione e poi mediata.	57
4.3	Travel time medio, max wait e travel time massimo sui preset di ablation, stessi dati delle Tabelle 4.6-4.7.	59
4.4	Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) sui preset di ablation, calcolata per singola configurazione/seed e poi mediata.	59
4.5	Travel time medio, max wait e travel time massimo sui meccanismi spaziali, stessi dati delle Tabelle 4.8-4.9.	63
4.6	Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) sui meccanismi spaziali, calcolata per singola configurazione/seed e poi mediata.	63

Elenco delle tabelle

3.1	Le 8 fasi semaforiche e i movimenti che ciascuna abilita.	20
3.2	Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.4.	22
3.3	Le cinque fasce del profilo di addestramento <i>complete</i> (totale: ~ 4.700 veicoli).	25
3.4	Tutte le combinazioni di griglia, spaziatura e profilo di traffico impiegate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo e il nome del file di configurazione corrispondente.	26
4.1	Preset di ablazione e sottosistema isolato da ciascuno. La reward custom è definita da Eq. (3.3), mentre la reward paper è definita da Eq. (3.18).	53
4.2	Travel time medio, massimo, max wait e trip completion dei tre controllori presenti in ogni studio di questo capitolo, mediati sulle 3 configurazioni di training.	54
4.3	Come Tabella 4.2, mediata sulle 7 configurazioni di test.	54
4.4	Travel time medio, massimo e max wait, mediati sulle 3 configurazioni di training.	56
4.5	Come Tabella 4.4, mediata sulle 7 configurazioni di test.	56
4.6	Travel time medio, massimo e max wait sui preset di ablazione, mediati sulle 3 configurazioni di training.	58
4.7	Come Tabella 4.6, mediata sulle 7 configurazioni di test.	58
4.8	Travel time medio, massimo e max wait sui meccanismi spaziali, mediati sulle 3 configurazioni di training.	61
4.9	Come Tabella 4.8, mediata sulle 7 configurazioni di test.	61

4.10	Percentuale di trip completion tc per ogni modello e ogni configurazione, corretta escludendo dal denominatore i veicoli strutturalmente impossibilitati ad arrivare in tempo. Nessun modello scende sotto l'80%; a parte Fixed-Time (baseline non adattiva) l'unico caso ricorrente sotto il 90% è MetaSTGCN 1L.	64
4.11	Travel time medio $\overline{tt}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.	65
4.12	Travel time massimo $tt_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.	66
4.13	Max wait $wait_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi. Sulle config 4x4-200, valori a piena corsia.	67

Acronimi e Notazione

Acronimi

Acronimo	Significato
MDP	Markov Decision Process
RL	Reinforcement Learning
DQN	Deep Q-Network
PER	Prioritized Experience Replay
BPTT	Backpropagation Through Time
GNN	Graph Neural Network
GCN	Graph Convolutional Network
GAT	Graph Attention Network
TSC	Traffic Signal Control
SMK	Spatial Meta-Knowledge (learner)
TMK	Temporal Meta-Knowledge (learner)
CS	Cooperation Spatial (modulo Meta-GAT)
CST	Cooperation Spatial-Temporal (modulo Meta-GAT)

Notazione principale

Simbolo	Significato
$\mathcal{S}, \mathcal{A}, P, R, \gamma$	Stati, azioni, transizione, ricompensa, sconto
π, V^π, Q^π	Politica, funzione valore-stato, funzione valore-azione
θ, θ^-	Pesi della rete online e della target network
τ	Coefficiente di soft update
α (in RL)	Esponente di prioritizzazione del PER
α (in questo lavoro)	Peso della penalità anti-starvation nella ricompensa
ε	Probabilità di esplorazione
s_i^t, a_i^t, r_i^t	Stato, azione, ricompensa dell'intersezione i al tempo t
n_i^t, w_i^t, p_i^t	Veicoli per corsia, attesa massima per corsia, fase corrente
d_0	Dimensione dello stato in ingresso
d_1	Dimensione nascosta del modello
H, d_h	Numero di teste di attenzione e dimensione per testa
e_i^t, e_j^t	Embedding temporale e spaziale del Dual State Encoder
x_i^t, h_i^t, c_i^t	Output, stato nascosto e stato di cella del Meta-LSTM
$\text{SMK}(i), \text{TMK}(i)$	Meta-embedding spaziale e temporale dell'intersezione i
$z_{\text{CS}}(i), z_{\text{CST}}(i)$	Output dei moduli Meta-GAT spaziale e spazio-temporale
$\bar{t}t, tt_{\max}$	Travel time medio e massimo
tc	Trip completion, percentuale di veicoli che completano il percorso
$\text{wait}_{\max}$	Max wait, tempo massimo di un veicolo sulla stessa corsia senza superare l'incrocio

Capitolo 1

Introduzione

Un incrocio semaforizzato stabilisce periodicamente quali flussi veicolari autorizzare e quali arrestare. Su una singola intersezione l'ottimizzazione del numero di veicoli che riescono ad attraversarla è banale: si concede la precedenza alla direzione con il maggior traffico in attesa. Tuttavia se si considera una rete composta da vari incroci e se si cerca inoltre di ottimizzare anche il tempo di attesa dei veicoli, il problema diventa molto più complesso. La fase selezionata da un incrocio influenza con un certo ritardo temporale il volume di veicoli in arrivo alle intersezioni adiacenti. Pertanto il controllo semaforico su rete si configura come un problema di coordinamento distribuito ancor prima che di ottimizzazione isolata.

Attualmente la stragrande maggioranza delle intersezioni è regolata da sistemi a tempo fisso. Tale approccio prevede una sequenza di fasi di durata predeterminata, calcolata una tantum sulla base di stime di traffico medie. Il sistema non reagisce quindi in alcun modo alle condizioni osservate in tempo reale. Al giorno d'oggi si possono comunque individuare alcuni casi in cui la scelta della fase semaforica viene influenzata dal traffico, come negli incroci che presentano un sensore di rilevamento di veicoli in prossimità del semaforo, grazie al quale per esempio si attiva il flusso sulle strade secondarie solo quando si presenta fisicamente un veicolo. Questi sistemi sono però molto limitati, sia dal punto di vista dei dati raccolti, sia dal punto di vista delle azioni eseguibili. La ricerca si è quindi concentrata verso lo studio di sistemi di controllo semaforico che potessero, almeno in parte, reagire più concretamente alle condizioni di traffico in tempo reale, necessitando però di una raccolta di dati più elevata. Al momento si possono trovare nel mondo e in Italia incroci che sfruttano i risultati di queste ricerche, ma la loro diffusione non è ancora ampia a causa di ostacoli economici, tecnici, burocratici e normativi.

Detto ciò, lo studio di questi approcci reattivi rimane ad ogni modo interessante e di potenziale beneficio. Uno degli algoritmi che hanno ottenuto maggiore

popolarità in questo ambito è MAX-PRESSURE (Varaiya, 2013), il quale introduce una regola locale che seleziona a ogni istante la fase con la pressione istantanea più alta (differenza tra veicoli in ingresso e in uscita sulle corsie servite da quella fase). MAX-PRESSURE garantisce proprietà di stabilità della rete nel suo insieme, ma la sua natura puramente greedy non offre alcuna garanzia di equità (*fairness*): in letteratura sul controllo semaforico, l’equità indica che tutti gli utenti della strada ricevano un accesso equo alle risorse dell’intersezione, evitando che alcuni veicoli o alcune direzioni sperimentino attese eccessive rispetto ad altri (Xiao et al., 2025) — un obiettivo concettualmente distinto dall’efficienza aggregata di rete (tempo di percorrenza medio, throughput): un controllore può massimizzare il flusso complessivo e allo stesso tempo lasciare una minoranza di veicoli sistematicamente penalizzata, se la sua regola di decisione non include alcun meccanismo che penalizzi esplicitamente le attese prolungate. È esattamente il caso di MAX-PRESSURE: un’intersezione posta su un’arteria ad alto scorrimento tende a favorire sistematicamente la direzione dominante, costringendo i flussi minoritari ad attese anche indefinitamente lunghe, poiché la regola sceglie sempre la fase a pressione istantanea più alta senza mai considerare da quanto tempo attende la direzione esclusa — una criticità che questo lavoro osserva empiricamente (Capitolo 3).

Alcuni metodi recenti in letteratura affrontano il problema penalizzando esplicitamente le fasi responsabili di attese eccessive, tipicamente solo oltre una soglia, per non sacrificare le prestazioni medie quando non è necessario: FELight, ad esempio, introduce una metrica di equità basata sui tempi di attesa dei veicoli e penalizza selettivamente le fasi con ritardi eccessivi, riportando una riduzione delle attese prolungate su direzioni specifiche senza compromettere l’efficienza complessiva della rete (Xiao et al., 2025). Questa tesi persegue lo stesso obiettivo con un meccanismo diverso, formalizzato nel Capitolo 3: un termine dedicato della ricompensa penalizza l’attesa massima osservata su una corsia non servita dalla fase corrente, pesato da un iperparametro che ne modula l’intensità — concettualmente lo stesso principio (penalizzare la coda peggiore, non solo ottimizzare il throughput aggregato), applicato però a ogni passo di decisione per ogni intersezione, non a un sottoinsieme di fasi identificate a priori. È il motivo per cui questo lavoro accetta, entro certi limiti, un tempo di percorrenza medio peggiore pur di ottenere un’attesa massima più contenuta: a parità di rete, un controllore che fa aspettare pochi veicoli molto a lungo non è preferibile a uno leggermente più lento in media ma che non lascia mai nessuno bloccato a tempo indefinito.

Per misurare quanto questo meccanismo funzioni nella pratica, questo lavoro non si affida al solo segnale di ricompensa — che dipende dai propri stessi parametri e

non è quindi un metro di giudizio imparziale — ma a una metrica puramente osservazionale, calcolata identicamente per ogni controllore indipendentemente da come è stato addestrato o se non è stato addestrato affatto: la massima attesa storica *consecutiva* osservata su una corsia, in una qualunque intersezione della rete, $\text{wait}_{\max}$ — cioè la durata del più lungo singolo stallo mai osservato, non un’attesa cumulativa sull’intero viaggio del veicolo. La metrica include deliberatamente gli stalli ancora in corso al termine dell’episodio, per lo stesso principio di non-esclusione del caso peggiore che vale anche per il tempo di percorrenza (definito formalmente nel Capitolo 4, §4.1): un veicolo fermo nello stesso punto dall’inizio del blocco fino all’ultimo istante simulato non farebbe mai scattare un nuovo record dopo la fine dell’episodio, e ignorare questi casi premierebbe indirettamente chi lascia un veicolo bloccato per sempre invece di penalizzarlo come merita. Il caso non è ipotetico: un’attesa di 1125 secondi registrata da MAX-PRESSURE in uno degli esperimenti di questo lavoro è stata verificata ricostruendo la traiettoria del veicolo responsabile direttamente dal replay grezzo della simulazione — il veicolo risultava fermo nello stesso identico punto per oltre mille secondi, quasi l’intera durata dell’episodio simulato.

L’obiettivo di questo lavoro è dunque duplice, e i suoi due termini non sono sempre allineati: da un lato ridurre il tempo di percorrenza medio e massimo dei veicoli (efficienza), dall’altro impedire che una direzione di traffico venga sistematicamente sacrificata a vantaggio delle altre (equità). Il Capitolo 4 mostra come questo compromesso si comporti concretamente al variare del peso assegnato alla componente di equità nella ricompensa.

A questi due obiettivi se ne affianca un terzo, di natura diversa dai primi due: verificare che le prestazioni misurate non dipendano dalla specifica rete stradale su cui il modello è stato addestrato, ma si mantengano, almeno in parte, su topologie e condizioni di traffico mai incontrate durante il training.

Gran parte della letteratura recente sul controllo semaforico appreso non affronta sistematicamente questa domanda: la prassi più diffusa è addestrare e valutare un modello sulla stessa rete stradale, al più ripetendo l’intero esperimento, addestramento incluso, su una rete diversa — non trasferendo un singolo modello già addestrato a una rete che non ha mai visto. CoLight (Wei et al., 2019) conduce cinque esperimenti indipendenti, due su reti sintetiche e tre su reti reali (Manhattan, Hangzhou, Jinan), ciascuno con il proprio addestramento; MetaSTGAT (Wang et al., 2022), il modello di riferimento di questo lavoro, segue lo stesso schema su una griglia sintetica e due dataset reali. Quando in questi lavori compare il termine *generalization*, indica quasi sempre la tenuta a diverse intensità di traffico sulla stessa rete, non a una topologia strutturalmente diversa (Wang et al., 2026; Sun

et al., 2026). Zhang et al. lo rendono esplicito nella motivazione di GeneraLight (Zhang et al., 2020): la maggior parte dei modelli RL per il controllo semaforico è addestrata e valutata nello stesso ambiente di traffico, il che porta a overfitting e a una scarsa capacità di generalizzazione quando le condizioni reali cambiano — un problema riconosciuto abbastanza da giustificare un lavoro dedicato a risolverlo, il che è di per sé indicativo di quanto la generalizzazione resti l’eccezione, non la norma.

Le eccezioni che esistono restano circoscritte a un’unica dimensione di variazione. AttendLight (Oroojlooy et al., 2020) addestra su un insieme eterogeneo di intersezioni singole e ne verifica le prestazioni su intersezioni mai viste, ma dichiara che l’estensione a una rete di più intersezioni coordinate resta lavoro futuro. Advanced-XLight (Zhang et al., 2022) misura un rapporto di trasferimento tra un modello addestrato su un dataset reale e valutato direttamente su un altro, senza ulteriore addestramento, ma sempre tra reti di scala paragonabile. Lo stesso GeneraLight generalizza al variare del flusso di traffico, non della topologia della rete.

Questo lavoro adotta invece la generalizzazione come requisito sistematico su un asse che la letteratura esaminata non copre: ogni modello, RL o baseline, è addestrato una sola volta su una griglia 4×4 e valutato, senza alcun riaddestramento, anche su topologie di dimensione maggiore mai incontrate (griglie 5×5 e 6×6) e su una spaziatura stradale diversa, oltre che su profili di traffico differenti da quello di addestramento (Capitolo 4, §4.2).

Il controllo appreso tramite *reinforcement learning* (RL) propone un paradigma differente a quello di MAX-PRESSURE: invece di una regola scritta a mano, una politica π viene addestrata a massimizzare una ricompensa che codifica l’obiettivo di rete (tipicamente throughput e tempo di viaggio) osservando ripetutamente le conseguenze delle proprie azioni in simulazione. Applicato a una singola intersezione, il problema è modellabile come un MDP standard; su scala di rete, assume la forma di un sistema multi-agente a osservabilità parziale, dove ogni nodo rileva esclusivamente il proprio intorno locale. Emerge quindi la necessità di un meccanismo per la condivisione delle informazioni tra agenti adiacenti. Le *graph neural network* (GNN) assolvono a questa funzione: rappresentando la rete stradale come un grafo, con le intersezioni come nodi e le strade come archi, l’informazione si propaga tra nodi vicini, permettendo a ogni intersezione di condizionare la propria decisione sullo stato dei suoi vicini immediati (e non) senza richiedere una politica centralizzata.

MetaSTGAT (Wang et al., 2022) combina questi aspetti con uno ulteriore: invece di condividere gli stessi pesi tra tutte le intersezioni della rete, un modulo di *meta-learning* genera dinamicamente i pesi di uno strato di attenzione e di uno stra-

to ricorrente a partire da caratteristiche locali dell’intersezione, così che nodi diversi possano essere serviti da trasformazioni diverse pur restando un unico modello addestrato end-to-end. Gli autori originali riportano una riduzione del travel time medio rispetto a CoLight (Wei et al., 2019), un metodo di riferimento basato su *graph attention* ma con pesi condivisi. La combinazione di *attention* su grafo, ricorrenza e reinforcement learning non è isolata: varianti di questo framework dominano la letteratura recente (Wang et al., 2026; Sun et al., 2026; Hu et al., 2026).

1.1 Contributo di questo lavoro

Questa tesi replica da zero l’architettura e la procedura di addestramento di MetaST-GAT, conducendo uno studio ablazionale sistematico e confrontandone le prestazioni con le baseline classiche del controllo semaforico. Nello specifico, i contributi sono:

- un ambiente di simulazione multi-intersezione costruito su CITYFLOW (Zhang et al., 2019), con reti stradali a griglia di dimensione 4×4 , 5×5 e 6×6 , otto fasi semaforiche per intersezione e una formulazione MDP dettagliata nel Capitolo 3;
- una generazione parametrica dei dati di traffico, con due livelli di densità (*flat* e *peak*); in ciascuna configurazione viene potenziato artificialmente il traffico su una riga della griglia simulando un’arteria stradale, in modo da verificare sistematicamente se un modello di controllo produce uno squilibrio direzionale quando il traffico non è simmetrico;
- un sistema di ablation che isola singolarmente ogni differenza tra l’implementazione proposta in questa tesi e la procedura descritta nell’articolo originale, organizzato in sei configurazioni che vanno dalla replica fedele del paper al modello più avanzato di questo lavoro;
- un protocollo di valutazione che riporta media e caso peggiore del tempo di viaggio dei veicoli, oltre alla massima attesa registrata da parte di un veicolo all’interno di un incrocio

Al momento della stesura di questo capitolo, l’estensione dell’architettura verso meccanismi spaziali alternativi alla *graph attention* (una convoluzione grafica standard, GCN, e un meccanismo di propagazione a lungo raggio ispirato a SONAR) è ancora in fase di progettazione e non è inclusa nei risultati qui presentati. Il Capitolo 3 ne introduce comunque la motivazione, e il confronto sperimentale a parità di resto dell’architettura è rimandato a un aggiornamento successivo di questo lavoro.

1.2 Struttura della tesi

Il Capitolo 2 introduce il background necessario a leggere i capitoli successivi senza assumere familiarità pregressa con nessuno dei tre campi coinvolti: reinforcement learning, graph neural network e controllo semaforico. Il Capitolo 3 formalizza il problema affrontato come MDP, descrive l’ambiente di simulazione e le baseline classiche, e presenta l’architettura di MetaSTGAT insieme a ogni differenza introdotta rispetto all’articolo originale, motivata caso per caso. Il Capitolo 4 descrive il protocollo sperimentale e riporta i risultati ottenuti, incluso lo studio di ablation. Il Capitolo ?? discute i risultati alla luce della domanda di ricerca, dichiara i limiti dello studio e indica il lavoro rimasto per la seconda parte del progetto.

Background

[illegible]
$$\begin{bmatrix} f_i^t \\ l_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(W_{\{f, l, o, g\}}^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f, l, o, g\}} \right), \quad c_i^t = f_i^t \odot c_i^{t-1} + l_i^t \odot g_i^t, \quad x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t) \quad (2.1)$$

dove $\mathcal{S}$ è lo spazio degli stati, $\mathcal{A}$ lo spazio delle azioni, $P(s' \mid s, a)$ la probabilità di transizione markoviana, $R(s, a)$ la ricompensa attesa e $\gamma \in [0, 1)$ il fattore di sconto, che pondera l'importanza delle ricompense future rispetto a quelle immediate. Un agente segue una politica $\pi(a \mid s)$, una distribuzione di probabilità sulle azioni

condizionata allo stato corrente; il suo obiettivo è massimizzare il ritorno atteso

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

ovvero la somma scontata delle ricompense future a partire dall'istante t . La funzione valore-stato $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ e la funzione valore-azione $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$ misurano rispettivamente quanto ritorno atteso si ottiene seguendo π da uno stato, o da una coppia stato-azione. Queste soddisfano l'equazione di Bellman, che esprime il valore di uno stato in funzione del valore atteso degli stati successivi:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V^\pi(s')] \quad (2.2)$$

Per una politica ottima π^* che massimizza $V^\pi(s)$ per ogni stato simultaneamente, vale l'analoga equazione di Bellman di ottimalità:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot \mid s, a)} \left[R(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.3)$$

Il Q-learning tabellare approssima Q^* calcolando iterativamente l'Equazione 2.3, ma richiede una tabella con tutte le coppie (s, a) , il che risulta inapplicabile per spazi di stato $\mathcal{S}$ continui o ad alta dimensionalità, come nel caso del controllo semaforico.

Deep Q-Network (DQN). Mnih et al. (2015) superano tale limite sostituendo la tabella con una rete neurale $Q(s, a; \theta)$. Inoltre, per mitigare l'instabilità dell'addestramento dovuta alla correlazione temporale dei campioni, introducono il *replay buffer*, ovvero una memoria da cui vengono campionati mini-batch di transizioni (s, a, r, s') in modo casuale, ripristinando l'assunzione di indipendenza necessaria per la discesa del gradiente stocastico.

Esplorazione ε -greedy. Un agente che seguisse sempre e solo l'azione a valore Q stimato più alto non raccoglierebbe mai esperienza sulle conseguenze di azioni alternative, rischiando di convergere su una politica sub-ottima per via di una mancata esplorazione. La strategia più comune per bilanciare sfruttamento (*exploitation*, scegliere l'azione ritenuta migliore) ed esplorazione (*exploration*, raccogliere informazione su azioni meno note) è ε -greedy: a ogni passo di decisione, con probabilità ε l'agente sceglie un'azione uniformemente a caso, e con probabilità $1 - \varepsilon$ sceglie $\operatorname{argmax}_a Q(s, a; \theta)$. Il valore di ε tipicamente decresce nel corso dell'addestramento, da un valore iniziale vicino a 1, dunque da un'esplorazione quasi totale, verso un

valore finale piccolo ma non nullo, dunque conducendo ad un comportamento quasi del tutto deterministico.

Estensioni usate in questo lavoro. Il DQN di base ammette diverse estensioni indipendenti, ciascuna delle quali corregge una specifica fonte di instabilità o inefficienza; la procedura di training descritta nel Capitolo 3 le adotta tutte, motivandole caso per caso rispetto all'articolo di riferimento:

- Il **Double DQN** (van Hasselt et al., 2016) disaccoppia la selezione dell'azione dalla sua valutazione, usando la rete online per scegliere l'azione migliore e una *target network* $Q(\cdot; \theta^-)$ solo per valutarla. Tale rete viene aggiornata periodicamente con i pesi della rete online e viene usata per calcolare il target $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$, in particolare $y = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$. Questo meccanismo evita il problema del *moving target*, il quale si verifica invece ottimizzando questa stima rispetto a parametri in continua variazione, e riduce il bias sistematico di sovrastima del DQN standard.
- Il **Prioritized Experience Replay** (PER, Schaul et al., 2016) sostituisce il campionamento uniforme dal replay buffer con un campionamento proporzionale all'errore di differenza temporale (*TD error*) δ_i , ovvero la discrepanza tra il ritorno futuro stimato e quello ottenuto. Assegnando a ciascuna transizione una priorità $p_i = |\delta_i| + \varepsilon$, la probabilità di campionamento della stessa è $P(i) = p_i^\alpha / \sum_k p_k^\alpha$, con $\alpha \in [0, 1]$ iperparametro che ne controlla la dipendenza dall'errore. Per correggere il bias introdotto da un campionamento non uniforme, la loss viene pesata tramite coefficienti di *importance sampling* $w_i = (N \cdot P(i))^{-\beta}$, dove N è la dimensione del buffer e $\beta \in [0, 1]$ iperparametro dinamico che ne controlla il grado di correzione.
- La **Huber loss** (o *smooth L1*) sostituisce l'errore quadratico medio (MSE) con una funzione quadratica vicino allo zero e lineare lontano da esso, risultando meno sensibile agli outlier prodotti da transizioni con errore di stima molto grande:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \delta \\ \delta \left(|y - \hat{y}| - \frac{1}{2}\delta \right) & \text{altrimenti} \end{cases} \quad (2.4)$$

Tipicamente $\delta = 1$.

- Il **soft update** (*Polyak averaging*) sostituisce l'aggiornamento periodico discreto della *target network* con una media mobile esponenziale:

$$\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$$

con $\tau \ll 1$, stabilizzando le variazioni del target a ogni passo di ottimizzazione.

- Quando la rete Q include un modulo ricorrente (ad esempio una LSTM come nel nostro caso, vedi Capitolo 3), l'assunzione di Markov sulle singole transizioni campionate dal *replay buffer* viene invalidata. Infatti, lo stato ricorrente (h, c) usato durante il training viene inizializzato a zero, introducendo un disallineamento rispetto allo stato generato sequenzialmente durante la raccolta dati: un problema noto come *hidden state staleness*. Seguendo Kapturowski et al. (2019), campioniamo intere sequenze di lunghezza L e dedichiamo i primi passi della sequenza (*burn-in*) esclusivamente a ricostruire uno stato ricorrente plausibile, senza calcolare gradiente, prima di eseguire la *backpropagation through time* (BPTT) sui passi restanti.

2.2 Graph Neural Network

Numerosi domini applicativi generano dati privi di una struttura regolare (come le immagini) o sequenziale (come il testo). Questi dati si organizzano spesso in topologie a grafo dove le relazioni spaziali veicolano informazione cruciale. Un esempio naturale di grafo è costituito da una rete stradale: le intersezioni rappresentano i nodi, mentre le strade che le collegano sono gli archi. Lo stato di un'intersezione (ad esempio, il numero di veicoli in attesa o la fase semaforica attiva) è influenzato da ciò che accade nelle intersezioni adiacenti. Le *Graph Neural Network* (GNN) estendono i paradigmi di convoluzione e attenzione a questi domini a grafo, implementando meccanismi di *message passing* tra nodi adiacenti.

Graph Convolutional Network (GCN). Kipf and Welling (2017) propongono la formulazione più basilare di propagazione su grafo tra quelle qui discusse: la rappresentazione di ciascun nodo si aggiorna mediando quelle dei vicini con un peso fisso, determinato unicamente dalla topologia locale, non dal contenuto delle rappresentazioni stesse. Indicando con $\hat{A} = A + I$ la matrice di adiacenza a cui è stato aggiunto un self-loop per ogni nodo (cosicché un nodo contribuisca anche al proprio aggiornamento) e con $\hat{D}$ la matrice diagonale dei gradi corrispondente ($\hat{D}_{ii} = \sum_u \hat{A}_{iu}$), la regola di propagazione di un singolo layer si scrive, in forma

matriciale:

$$H^{(l+1)} = \sigma\left(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)}\right) \quad (2.5)$$

dove $H^{(l)} \in \mathbb{R}^{I \times d_l}$ raccoglie le rappresentazioni di tutti gli I nodi del grafo al layer l , $W^{(l)}$ è la matrice di pesi condivisa da ogni nodo e σ una non linearità puntuale. Per un singolo nodo i , la normalizzazione simmetrica $\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$ equivale a pesare il contributo di ogni vicino u con $1/\sqrt{\hat{d}_i \hat{d}_u}$:

$$h'_i = \sigma\left(\sum_{u \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W h_u\right) \quad (2.6)$$

un peso identico per ogni arco del vicinato di i , funzione solo del grado dei due nodi coinvolti e mai del loro contenuto. È, per costruzione, il termine di paragone più naturale rispetto a cui misurare cosa aggiunge un meccanismo di attenzione appreso come quello discusso di seguito.

Graph Attention Network (GAT). Veličković et al. (2018) sostituiscono il peso fisso della GCN con un coefficiente di attenzione appreso, specifico per ciascuna coppia di nodi collegati: il contributo di un vicino non dipende più soltanto dalla topologia, ma anche dal suo contenuto. Dato un nodo i con vicinato $\mathcal{N}(i)$, embedding di input h_i , matrice di proiezione condivisa W e vettore di attenzione $\vec{a}$:

$$e_{iu} = \text{LeakyReLU}\left(\vec{a}^\top [W h_i \parallel W h_u]\right), \quad \alpha_{iu} = \frac{\exp(e_{iu})}{\sum_{u' \in \mathcal{N}(i)} \exp(e_{iu'})} \quad (2.7)$$

$$h'_i = \sigma\left(\sum_{u \in \mathcal{N}(i)} \alpha_{iu} W h_u\right) \quad (2.8)$$

dove $\parallel$ denota la concatenazione. Con K teste di attenzione indipendenti, gli output si concatenano nei layer intermedi e si mediano nell'ultimo, secondo lo schema multi-head introdotto per l'attenzione da Vaswani et al. (2017). A differenza della GCN, il coefficiente α_{iu} consente a un nodo di ponderare diversamente i propri vicini in base al loro contenuto, non solo alla loro posizione nel grafo: una proprietà rilevante per un'intersezione i cui vicini abbiano importanza marcatamente diversa, come nel caso di un'arteria principale contrapposta a una strada secondaria. La funzione di compatibilità additiva appena descritta, per quanto la più nota, non è l'unica possibile: il Capitolo 3 (Meta-GAT) ne impiega una variante strutturalmente diversa, con Query, Key e Value esplicite e distinte al posto della proiezione W condivisa, e un prodotto scalare al posto della concatenazione.

Reti spatio-temporali su grafo. Una rete stradale non è soltanto un grafo: è un grafo il cui stato (occupazione delle corsie, fase attiva) evolve nel tempo, e le due dimensioni risultano intrecciate, poiché il valore osservato a un nodo in un dato istante è correlato sia con i valori passati allo stesso nodo sia con quelli ai nodi limitrofi. La famiglia di modelli che combina un layer di propagazione sul grafo (GCN o GAT) con una componente ricorrente o convoluzionale sull’asse temporale prende il nome di *spatiotemporal graph neural network* (STGNN), un termine ormai consolidato nella letteratura sul forecasting di serie temporali multivariate correlate (Cini et al., 2025). MetaSTGAT, descritto nel dettaglio nel Capitolo 3, costituisce un esempio di questa famiglia applicata al controllo semaforico anziché alla previsione: un layer di *graph attention* cattura la dipendenza spaziale tra intersezioni limitrofe, un layer ricorrente (LSTM) ne cattura l’evoluzione temporale, e i due vengono addestrati congiuntamente end-to-end.

Pesi generati da meta-learning. Sia la GCN sia la GAT, nella loro formulazione originale, condividono gli stessi pesi su tutti i nodi del grafo: un’assunzione ragionevole quando i nodi sono sufficientemente omogenei, meno quando, come in una rete stradale reale, la topologia locale di ciascun nodo (numero di strade entranti, presenza di un’arteria, posizione periferica o centrale) può differire sensibilmente da nodo a nodo. Un *hypernetwork* affronta il problema generando i pesi di un layer non come parametri fissi appresi direttamente, bensì come output di una seconda rete condizionata sulle caratteristiche locali del nodo: il Capitolo 3 descrive nel dettaglio come MetaSTGAT applichi questo principio sia al layer di *graph attention* sia al layer ricorrente.

Propagazione a lungo raggio. Sia la GCN sia la GAT propagano informazione a un solo passo di distanza (un arco nel grafo) per layer: raggiungere un vicino a distanza k richiede impilarne altrettanti, con un costo di parametri crescente e un rischio noto di *over-smoothing*, per il quale le rappresentazioni dei nodi divengono troppo omogenee. Meccanismi più recenti, tra cui **SONAR** (Trenta et al., 2025), affrontano il problema della propagazione a lungo raggio secondo un principio diverso, ispirato a sistemi dinamici oscillatori piuttosto che alla concatenazione sequenziale di layer: il dettaglio di questo meccanismo e la sua applicazione al controllo semaforico sono rimandati al Capitolo 3.

2.3 Traffic Signal Control come problema

Il controllo semaforico (*Traffic Signal Control*, TSC) si articola in tre approcci, in ordine crescente di complessità.

Controllo a tempi fissi. Una sequenza di fasi di durata predeterminata, calcolata su stime di traffico medie, si ripete ciclicamente senza mai osservare lo stato reale della rete. È un approccio robusto, in quanto non può fallire per un sensore guasto, ma per costruzione non reattivo: la stessa sequenza viene eseguita indipendentemente da congestioni anomale, incidenti o variazioni del traffico non modellate in fase di pianificazione.

Controllo reattivo. Una regola locale osserva lo stato corrente dell'intersezione (tipicamente il numero di veicoli in coda per corsia) e sceglie l'azione che ottimizza un criterio istantaneo. MAX-PRESSURE (Varaiya, 2013) ne è l'esponente più studiato: modellando il movimento dei veicoli come una rete di code *store-and-forward*, definisce la pressione di una fase ϕ come la differenza tra veicoli sulle corsie che quella fase servirebbe in ingresso e in uscita, e sceglie sempre l'azione a pressione massima. Il risultato teorico centrale dell'articolo originale è che Max-Pressure è *massimamente stabilizzante*; ciò significa che se è possibile stabilizzare la rete, ovvero non far divergere la lunghezza delle code nel tempo, Max-Pressure la stabilizza. L'algoritmo garantisce questo risultato usando solo la lunghezza delle code sulle corsie adiacenti all'intersezione, e senza conoscere a priori il volume di domanda in ingresso. Max-Pressure è tuttavia miope per costruzione: non pianifica alcun compromesso su un orizzonte temporale non immediato, e la condizione di stabilità non tiene in considerazione le attese tra le diverse direzioni servite da un'intersezione, un aspetto di equità che questo lavoro esamina esplicitamente (Capitolo 4).

Controllo appreso. L'approccio basato sul *reinforcement learning* consente l'ottimizzazione diretta di metriche di rete globali (es. *travel time* o *throughput*) sostituendo euristiche come la pressione istantanea, al netto del costo computazionale dell'addestramento simulato. Applicato a una rete di più intersezioni, il problema si formalizza come un Processo Decisionale di Markov decentralizzato a osservabilità parziale (Dec-POMDP): ogni intersezione osserva solo il proprio stato locale, ma le sue azioni influenzano, con un ritardo proporzionale alla distanza spaziale, lo stato delle intersezioni adiacenti alterando il flusso di veicoli che le attraversano.

Architetture di coordinamento. Esistono tre strategie distinte che affrontano la dipendenza tra intersezioni vicine. Un agente centralizzato che decide le azioni di tutta la rete con un'unica politica elimina il problema per costruzione, ma lo spazio delle azioni congiunte cresce esponenzialmente con il numero di intersezioni ed è im-

praticabile oltre poche decine di nodi. All'estremo opposto, un agente indipendente per intersezione, che non condivide alcuna informazione con i propri vicini; questo approccio scala senza difficoltà ma tratta ogni intersezione come se fosse isolata, ignorando che le sue azioni si propagano alla rete circostante. La via intermedia, adottata anche da questo lavoro, fa comunicare agenti indipendenti condividendo informazione locale. La difficoltà, discussa da [Wei et al. \(2019\)](#), risiede nel fatto che la trasmissione dello stato dei vicini in una sequenza predefinita impone a tutte le intersezioni della rete gli stessi pesi interpretativi per ciascuna posizione della lista. Questo approccio non tiene conto della reale importanza che ogni specifico nodo adiacente può rivestire per le diverse intersezioni del sistema. Una GNN evita il problema: pesando il contributo di ogni vicino in base al suo contenuto invece che alla sua posizione in un elenco, permette a un unico insieme di pesi condiviso di generalizzare tra intersezioni con topologie locali diverse, come quelle della rete adottata in questo lavoro (Capitolo 3).

Coordinamento oltre l'adiacenza fisica. Il grafo usato da MetaSTGAT e da questo lavoro coincide con l'adiacenza fisica della rete stradale: un'intersezione comunica solo con le intersezioni immediatamente collegate da una strada. Dalla letteratura si osserva che interazioni tra sole intersezioni adiacenti non siano sufficienti a catturare tutte le dinamiche di traffico e si propongono diverse soluzioni per ovviare a questo limite. In questo lavoro si è deciso di non affrontare questo tema, esplorandolo solo in parte nel Capitolo 4. Il problema resta comunque aperto e framework che combinano ingredienti proposti da MetaSTGAT, quali *graph attention*, una componente ricorrente sul tempo e reinforcement learning per il controllo semaforico, continuano ad essere oggetto di ricerca.

Una tassonomia delle soluzioni recenti. La letteratura più recente sul reinforcement learning applicato al TSC classifica i metodi lungo assi in parte indipendenti da quelli discussi finora. Un primo asse distingue, all'interno del multi-agente, due architetture di coordinamento: l'*addestramento centralizzato con esecuzione decentralizzata* (CTDE), in cui gli agenti condividono informazione globale solo durante il training per poi decidere localmente, e l'*addestramento ed esecuzione entrambi decentralizzati* (DTDE), in cui l'informazione resta sempre e solo locale, in entrambe le fasi. Un secondo asse raccoglie le tecniche di apprendimento integrate nel ciclo RL, ciascuna introdotta per rispondere a un limite specifico ([Xiao et al., 2025](#)): reti neurali a grafo per ottenere una migliore coordinazione tra intersezioni, componenti ricorrenti per catturare la dipendenza tra passi successivi a livello temporale, transfer-learning per generalizzare rapidamente a scenari nuovi, meta-learning per

adattarsi a condizioni operative diverse, reti generative avversarie per compensare dati di stato mancanti o incompleti. Nessuna di queste tecniche risolve da sola il problema: la letteratura le introduce quasi sempre in risposta a un limite preciso.

Dove si colloca MetaSTGAT. Il modello di questo lavoro è la combinazione di alcune delle tecniche appena citate. La sua è un'architettura DTDE con scambio locale di informazione, che affianca al *graph attention* (GAT) un componente ricorrente (LSTM) per catturare la dipendenza temporale oltre a quella spaziale, collocandosi nella famiglia delle reti spatio-temporali su grafo (STGNN); il modello sfrutta inoltre il meta-learning, con l'obiettivo di adattare dinamicamente la propria architettura alle condizioni della rete.

Simulazione. Valutare una politica di controllo semaforico su una rete reale non è praticabile durante lo sviluppo. Affinchè i risultati siano informativi, serve un simulatore che riproduca la dinamica del traffico con un dettaglio sufficiente, pur mantenendo un costo computazionale accettabile per l'addestramento di centinaia di episodi. CITYFLOW (Zhang et al., 2019) è il simulatore scelto in questo lavoro: rispetto ad alternative più diffuse come SUMO (Lopez et al., 2018), privilegia la velocità di simulazione rispetto al realismo microscopico del singolo veicolo, un compromesso adatto quando l'obiettivo primario è addestrare un agente su un numero elevato di episodi piuttosto che riprodurre con precisione millimetrica la dinamica di un incrocio specifico. Il Capitolo 3 descrive nel dettaglio come questo lavoro configura l'ambiente CityFlow.

Capitolo 3

Metodologia

Questo capitolo formalizza il controllo semaforico multi-intersezione come processo decisionale di Markov (§3.1), descrive l'ambiente di simulazione costruito su CITYFLOW (§3.2) e le baseline classiche a cui questa tesi confronta i propri modelli (§3.3). Presenta poi nel dettaglio l'architettura MetaSTGAT (§3.4), ogni differenza introdotta rispetto all'articolo di riferimento (§3.5), i meccanismi spaziali alternativi confrontati nel Capitolo 4 (§3.6) e la procedura di addestramento adottata (§3.7).

3.1 Formalizzazione del problema

Il Capitolo 2.1 introduce l'MDP come la tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ e il Capitolo 2.3 la sua estensione a una rete di I intersezioni come un Dec-POMDP a osservabilità locale: ogni intersezione i osserva solo il proprio stato, ma le sue azioni influenzano, con un ritardo proporzionale alla distanza spaziale, lo stato delle intersezioni adiacenti. Questa sezione istanzia ciascuna componente di quella tupla per il sistema di questo lavoro. Ad ogni istante t , l'intersezione i osserva uno stato $s_i^t \in \mathcal{S}$, sceglie un'azione $a_i^t \in \mathcal{A}$ e riceve una ricompensa r_i^t , mentre l'informazione viene propagata secondo la topologia del grafo stradale.

Stato ($\mathcal{S}$). Ogni intersezione ha 4 strade in ingresso, una per direzione cardinale, con 3 corsie per senso di marcia, per un totale di 12 corsie in ingresso e altrettante in uscita. A questa configurazione si aggiungono 8 fasi semaforiche selezionabili.

Lo stato è la concatenazione di tre componenti:

$$s_i^t = [n_i^t \parallel w_i^t \parallel p_i^t] \in \mathbb{R}^{d_0} \quad (3.1)$$

dove $n_i^t \in [0, 1]^{12}$ è il numero di veicoli per corsia in ingresso entro il raggio di visibilità d_{vis} (normalizzato dividendo per 30 e poi limitato tra 0 e 1), $w_i^t \in [0, 1]^{12}$ è il vettore dei tempi di attesa dei veicoli da più tempo in corsia ($\tanh(\frac{\text{attesa}}{300})$) e $p_i^t \in \{0, 1\}^8$ è la fase corrente in codifica one-hot, per un totale $d_0 = 32$ ($d_0 = 20$ se w_i^t è esclusa, come nel preset che riproduce il modello di riferimento, §3.7). Il campo visivo ristretto e la componente w_i^t sono due modifiche di questo lavoro rispetto al modello di riferimento, discusse in §3.5. Da un punto di vista pratico tutte queste feature possono essere calcolate tramite sensori stradali posti al punto di stop dell'incrocio e a distanze d_{vis} da esso.

Azione (A). $a_i^t \in \{0, \dots, 7\}$ seleziona una delle otto fasi che verranno descritte in §3.2; La transizione di "tutto-rosso" è gestita internamente dal simulatore e non appartiene allo spazio delle azioni. Un vincolo *anti-starvation* (che contrasta cioè l'eccessiva attesa dei veicoli all'incrocio) rappresenta una modifica di questo lavoro discussa in §3.5. Tale vincolo invalida la fase corrente come azione se è già stata selezionata due volte consecutive, forzando la rotazione:

$$\mathcal{A}_i^t = \{0, \dots, 7\} \setminus \{a_i^{t-1}\} \quad \text{se } a_i^{t-1} = a_i^{t-2}, \quad \mathcal{A}_i^t = \{0, \dots, 7\} \text{ altrimenti} \quad (3.2)$$

Transizione (P). Essendo l'algoritmo di addestramento (Double DQN) *model-free*, le probabilità di transizione $P(s' \mid s, a)$ non vengono modellate esplicitamente, ma campionate empiricamente. L'ambiente CITYFLOW è per natura deterministico, pertanto, fissati stato e azione, la transizione collassa su un singolo stato successivo con probabilità 1.

Fattore di sconto (γ). In questo lavoro viene fissato $\gamma = 0.85$ per ogni modello, senza alcuna ottimizzazione iperparametrica dedicata.

Ricompensa (R). La ricompensa usata dal modello di questo lavoro è:

$$r_i^t = \text{clip} \left(\frac{-(\text{Incoming}_i^t - \text{Passed}_i^t) - \alpha \cdot \text{MaxRedWait}_i^t - \text{Penalty}_i^t}{100}, -2, 0 \right) \quad (3.3)$$

dove Incoming_i^t è il numero di veicoli entrati nella finestra visiva dell'intersezione al tempo t , Passed_i^t il numero di veicoli che hanno lasciato la finestra visiva tra t e $t+1$, MaxRedWait_i^t il tempo di attesa massimo tra tutti i veicoli che si trovano nelle corsie con segnale rosso in ingresso all'istante t , $\text{Penalty}_i^t = 50$ se

$\text{Passed}_i^t = 0 \wedge \text{Incoming}_i^t > 0$ e 0 altrimenti, e $\alpha > 0$ l'iperparametro con cui si modula la penalizzazione per lo *starvation* (l'eccessiva attesa dei veicoli). Il fattore

di scala $1/100$ è introdotto per stabilizzare i gradienti. Per il calcolo di tutte queste metriche ciascun modello fa riferimento, se impostata, alla distanza d_{vis} .

Topologia di Rete (G). Le I intersezioni sono rappresentate dai nodi di un grafo $G = (V, E)$. Un arco orientato $(i, j) \in E$ esiste se un segmento stradale connette fisicamente i a j . A ogni nodo è inoltre aggiunto un arco verso se stesso (*self-loop*), cosicché i layer di *graph attention* possano pesare tanto lo stato del nodo stesso quanto quello dei vicini. Sotto il profilo della generalizzazione spaziale, la presente architettura impone che lo stato $s_i^{(t)}$ e le *meta feature* processino unicamente dati locali, senza alcun accesso all'identità del nodo i all'interno della rete. Conseguentemente, la policy non può prendere decisioni in base alla posizione dell'incrocio o al suo ruolo all'interno della rete (es. questo incrocio rappresenta un'arteria), ma è costretta ad apprendere euristiche reattive alle condizioni locali osservate.

3.2 Ambiente di simulazione

L'ambiente di simulazione si basa su CITYFLOW (Zhang et al., 2019), un simulatore microscopico open source con motore in C++ e binding in Python, progettato specificamente per il controllo semaforico su larga scala tramite reinforcement learning. La necessità di un nuovo strumento, come argomentato dagli autori originali, nasce dai limiti prestazionali di SUMO (Lopez et al., 2018), il simulatore open source più diffuso in letteratura. SUMO risulta infatti computazionalmente proibitivo per l'addestramento di agenti RL su reti di grandi dimensioni e ad alta densità veicolare. CITYFLOW garantisce una velocità di simulazione circa 20 volte superiore rispetto a SUMO su griglie 30×30 , ma il compromesso è un dettaglio minore, il che si rispecchia su una dinamica veicolare semplificata. Tale trade-off risulta ottimale in questo contesto, dove il vincolo critico è l'esplorazione dello spazio delle policy su migliaia di episodi, piuttosto che la fedeltà microscopica della singola traiettoria.

3.2.1 Struttura della rete e mappatura delle fasi

La topologia della rete è codificata in un file di configurazione (*roadnet*), che definisce l'insieme delle intersezioni (nodi) e dei segmenti stradali (archi) con le relative molteplicità di corsia. Per ogni intersezione il *roadnet* elenca le fasi semaforiche disponibili, ciascuna definita come l'insieme dei movimenti (coppie corsia di origine e di destinazione) abilitati in quella fase. L'azione dell'agente si riduce pertanto alla selezione dell'indice corrispondente alla fase da attivare. Ogni intersezione di questo lavoro modella 12 corsie in ingresso (quattro direzioni cardinali, tre corsie ciascuna: sinistra,

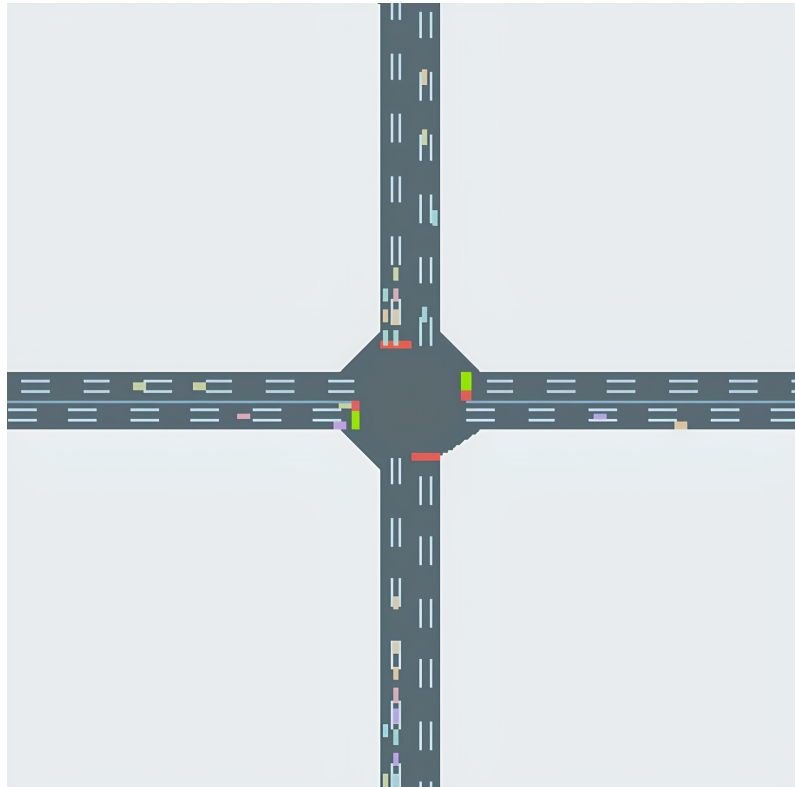


Figura 3.1: Rendering del visualizzatore di CITYFLOW per una singola intersezione a quattro vie: le indicazioni verde/rosso ai margini dell'incrocio mostrano i movimenti abilitati dalla fase corrente, i rettangoli colorati sulle corsie i veicoli in transito o in coda.

diritto, destra) e 8 fasi semaforiche selezionabili. La tabella 3.1 riporta la mappatura di tali fasi, adottando la nomenclatura N/S/E/W (*North/South/East/West*) e T/L (*Through*, diritto, e *Left*, sinistra).

Fase	Nome	Movimenti abilitati
0	NTST	Nord diritto+destra, Sud diritto+destra
1	NLSL	Nord sinistra, Sud sinistra
2	NTNL	Nord sinistra+diritto+destra
3	STSL	Sud sinistra+diritto+destra
4	WTET	Ovest diritto+destra, Est diritto+destra
5	WLEL	Ovest sinistra, Est sinistra
6	ETEL	Est sinistra+diritto+destra
7	WTWL	Ovest sinistra+diritto+destra

Tabella 3.1: Le 8 fasi semaforiche e i movimenti che ciascuna abilita.

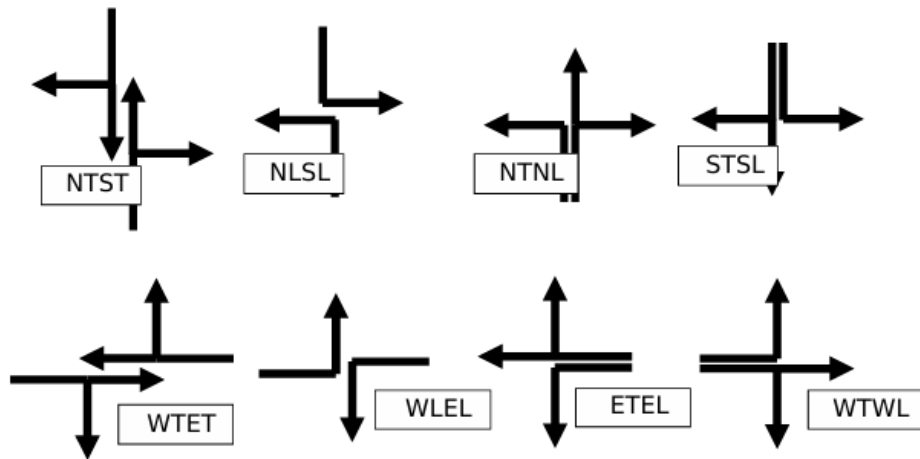


Figura 3.2: Rappresentazione grafica delle 8 fasi di Tabella 3.1: le frecce indicano i movimenti abilitati da ciascuna fase.

Il paper originale permette ai veicoli che svoltano a destra di muoversi liberamente, senza attendere il semaforo verde, a patto che non vi siano veicoli provenienti da altre direzioni. Al fine di prevenire una qualunque possibile interferenza tra diversi flussi e garantire un controllo più rigoroso, si è deciso di non adottare questo comportamento, permettendo la svolta a destra solo quando esplicitamente autorizzata dalla fase semaforica. L'insieme delle fasi sopra riportate rappresenta una configurazione tale da sfruttare al massimo lo spazio delle direzioni possibili, evitando al contempo conflitti tra le traiettorie dei veicoli.

3.2.2 Gestione dell'interverde

L'interverde è il periodo di tempo che intercorre tra la fine del verde di una fase semaforica e l'inizio del verde di quella successiva. Il suo scopo è quello di garantire la sicurezza stradale, permettendo ai veicoli che si trovano nell'intersezione di sgombrarla prima che venga attivata la fase successiva. Il motore di simulazione CITYFLOW non modella nativamente tale fase transitoria, nello specifico, non gestisce i semafori gialli e il conseguente passaggio sicuro tra due fasi consecutive; dunque l'interverde deve essere costruito esplicitamente. In questo lavoro, l'intervallo decisionale dell'agente (lo *step* di simulazione) ha una durata complessiva $\Delta t_{\text{step}} = 15$ s simulati, idealmente ripartita in tre sotto-fasi: verde ($\Delta t_{\text{green}} = 10$ s), giallo ($\Delta t_{\text{yellow}} = 3$ s) e "tutto-rosso" ($\Delta t_{\text{red}} = 2$ s). Per il limite di CITYFLOW sopra descritto e in questo studio accettato come compromesso, l'ambiente assimila il giallo al verde senza imporre modelli di decelerazione specifici, mantenendo pertanto i movimenti autorizzati dalla policy per una durata totale di $\Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13$ s consecutivi. Al termine di questa finestra temporale, viene eseguita per $\Delta t_{\text{red}} = 2$ s una fase che non è presente tra le azioni selezionabili dal modello, ovvero il semaforo rosso in contemporanea su tutte le corsie. Questo intervallo di guardia è necessario per garantire lo svuotamento dell'intersezione prima del cambio di fase, così da prevenire collisioni. A questo punto lo step giunge al termine e il modello formula una nuova decisione, scegliendo quale delle 8 fasi implementare nel ciclo successivo.

È importante sottolineare che questa accortezza riduce di fatto il tempo effettivo utile di decisione del singolo agente di controllo semaforico, in quanto due dei quindici secondi di ogni step sono dedicati alla fase "tutto-rosso" e quindi da considerarsi tempo morto (rappresentando circa il 13% del tempo totale).

3.2.3 Dataset e generazione parametrica

Ogni roadnet e ogni flusso di traffico usato in questo lavoro è generato parametricamente e in modo interamente deterministico da uno script dedicato, senza alcun campionamento empirico. Questo approccio consente di avere un pieno controllo sulle variabili operative come la struttura della rete, la densità veicolare e presenza di arterie principali, oltre a garantire l'esatta riproducibilità degli scenari di simulazione, un requisito metodologico fondamentale per un equo confronto tra modelli di controllo semaforico.

Topologie e spaziature. L'ambiente di simulazione implementa tre topologie a griglia regolare di dimensioni 4×4 (16 intersezioni), 5×5 (25 intersezioni) e

6×6 (36 intersezioni), impiegate per l'addestramento e per valutare la capacità di generalizzazione dei modelli rispetto alla scala spaziale.

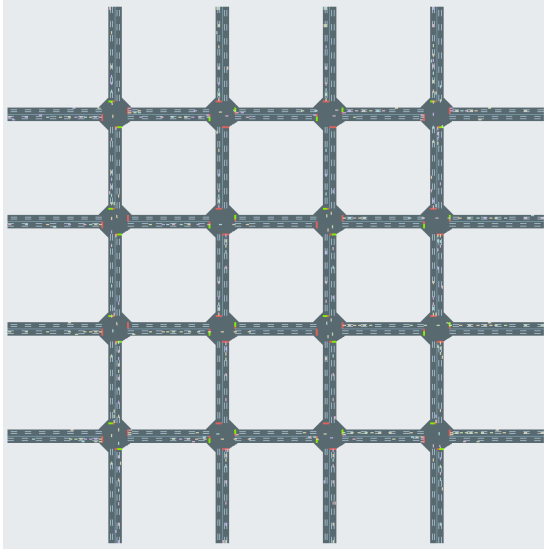
Un obiettivo centrale di questo lavoro consiste nell'indagare le capacità di coordinamento implicito apprese dalle diverse policy di controllo. In quest'ottica, assume particolare rilevanza la comparsa spontanea della così detta onda verde, uno standard strutturale nell'ingegneria del traffico contemporanea. La spaziatura tra incroci è pertanto di due tipi: calibrata per l'onda verde e non calibrata. In assenza di offset temporali, in quanto le transizioni di fase sono sincrone per tutti i nodi della rete, l'unico modo per ottenere un'onda verde, ovvero un flusso di veicoli che si propagano attraverso la rete senza fermarsi ai semafori, è calibrare la distanza fisica tra gli incroci sulla velocità di marcia e sulla durata del ciclo: alla velocità di riferimento $v_{\text{urb}} = 11.1 \text{ m/s}$, in un ciclo di $\Delta t_{\text{step}} = 15 \text{ s}$ viene percorsa una distanza pari a 166.5 m. Dunque la prima spaziatura proposta è proprio di questa misura, ma ne faremo riferimento con l'etichetta "100m". La spaziatura non calibrata invece è 100m più estesa e viene indicata con "200m".

Le roadnet non condividono tutte lo stesso ruolo. La griglia 4×4 con spaziatura "100m" compare sia in training sia in test, mentre le altre tre combinazioni topologia-spaziatura restano riservate esclusivamente al test e non vengono mai incontrate durante l'addestramento (Tabella 3.2). Questa separazione serve a isolare due assi di generalizzazione: le griglie 5×5 e 6×6 misurano la tenuta della policy su reti più estese di quella di addestramento, con più intersezioni da coordinare, mentre la spaziatura "200m" applicata alla griglia 4×4 misura la tenuta su una topologia mai vista con incroci più distanziati, priva della calibrazione per l'onda verde descritta sopra.

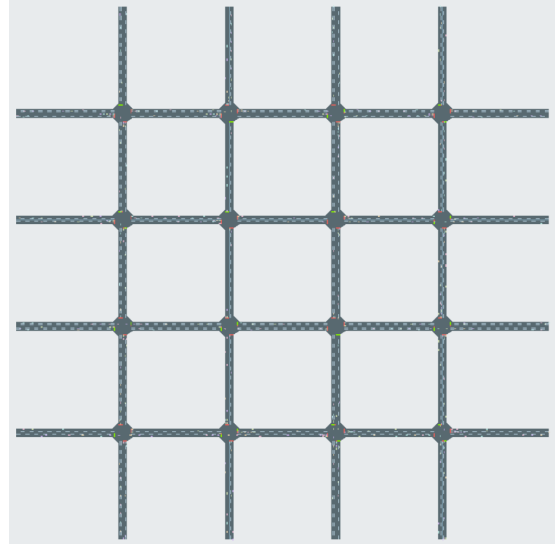
Griglia	Spaziatura	Utilizzo
4×4	100m	Train, Validation, Test
4×4	200m	Test
5×5	100m	Test
6×6	100m	Test

Tabella 3.2: Utilizzo di ciascuna combinazione topologia-spaziatura. Il dettaglio del confine tra training e validazione sulla griglia 4×4 a 100m è in Tabella 3.4.

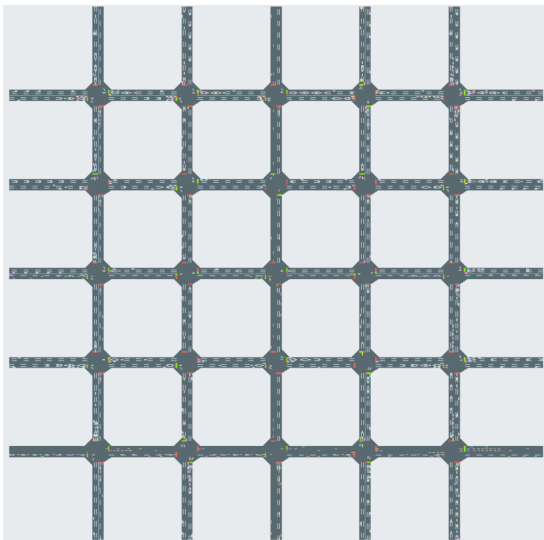
Percorsi veicolari. Il percorso di ciascun veicolo attraverso la rete non è fissato a priori, ma generato casualmente per ciascun punto di ingresso sul perimetro della griglia: a ogni incrocio attraversato, il percorso prosegue dritto con probabilità 0.60, svolta a destra con probabilità 0.30 o svolta a sinistra con probabilità 0.10, finché non esce dai confini della rete o non ripercorrerebbe un incrocio già attraversato (evitando



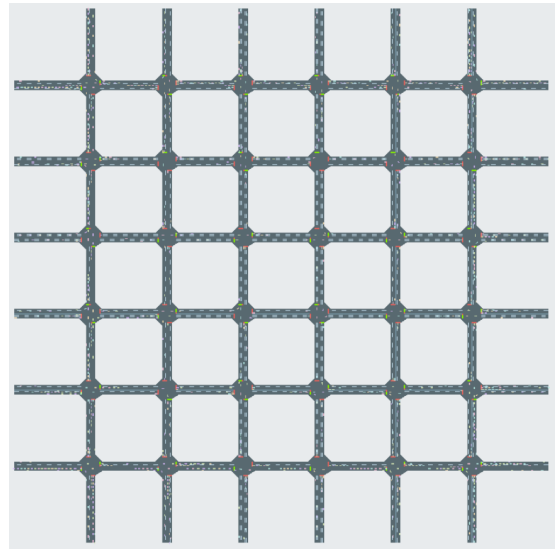
(a) 4×4 , 100m



(b) 4×4 , 200m



(c) 5×5 , 100m



(d) 6×6 , 100m

Figura 3.3: Rendering nel visualizzatore di CITYFLOW delle quattro combinazioni topologia-spaziatura di Tabella 3.2.

così percorsi ciclici). La probabilità complessiva di un percorso è il prodotto delle probabilità delle singole scelte che lo compongono; il numero di veicoli assegnato a ciascun percorso è proporzionale al suo peso relativo sul totale dei percorsi generati da quel punto di ingresso, calibrato per raggiungere il volume di traffico target di ciascuna fascia (Tabella 3.3).

Profili di traffico. Per ciascuna topologia sono stati generati tre profili di traffico: un flusso a intensità costante (*flat*), un profilo a gradino (*peak*, diviso in tre fasi temporali con un raddoppio del carico nel terzo centrale) e un profilo dinamico (*complete*) che simula il traffico di una giornata lavorativa (dettagliato nel seguito), impiegato esclusivamente in fase di addestramento. Ogni episodio ha una durata di $T_{\max} = 1800$ s simulati, corrispondenti a $1800 \text{ s} / \Delta t_{\text{step}} = 120$ cicli decisionali per agente.

I volumi dei veicoli immessi nella rete per ciascuna configurazione topologica sono stati determinati tramite una calibrazione empirica, così da garantire un livello di saturazione comparabile tra reti di dimensioni diverse e quindi una difficoltà del task di controllo invariata rispetto la scala topologica. In particolare la densità di ogni topologia è stata bilanciata affinché il *trip completion* ottenuto dalla baseline MAX-PRESSURE risultasse omogeneo (90%–94%).

Profilo di addestramento *complete*. A differenza degli scenari di validazione e test, caratterizzati da densità costanti o a singolo picco, il profilo di addestramento sintetizza l'evoluzione della domanda veicolare di un'intera giornata solare (24 ore) all'interno di un singolo episodio di 1800 s. Come dettagliato nella Tabella 3.3, il flusso è diviso in cinque fasce a intensità variabile: due fasce a basso carico (assimilabili agli orari notturni), due picchi di massima domanda (flussi di traffico di inizio e fine turno lavorativo) e un intervallo di raccordo centrale a regime moderatamente sostenuto. Questa marcata variazione di domanda veicolare all'interno di un singolo episodio è funzionale poiché costringe il modello ad adattare dinamicamente il piano di controllo, senza convergere verso una policy specializzata soltanto su un unico flusso di traffico.

Stocasticità e sbilanciamento direzionale. Per mitigare il rischio di *overfitting* e garantire la robustezza del modello, la generazione del profilo di addestramento integra due accorgimenti.

Innanzitutto, considerata la natura deterministica di CITYFLOW, addestrare un modello su una singola configurazione esporrebbe l'agente all'apprendimento mnemonico di un'esatta sequenza di veicoli. Per prevenire questa eventualità, si generano

Fascia	Intervallo simulato	Step di decisione	Veicoli/secondo
Flat basso	0-525s	0-35	1.2
Peak basso	525-675s	35-45	3.5
Flat alto	675-1200s	45-80	2.5
Peak alto	1200-1350s	80-90	4.2
Flat basso	1350-1800s	90-120	1.2

Tabella 3.3: Le cinque fasce del profilo di addestramento *complete* (totale: ~ 4.700 veicoli).

con diversi *seed* tre profili di traffico differenti, in modo da perturbare la distribuzione dei percorsi e gli istanti di immissione, pur mantenendo la medesima struttura generale descritta in Tabella 3.3; tali varianti vengono quindi alternate ciclicamente durante l'addestramento. Anche le configurazioni di validazione e test, su cui i modelli vengono valutati metodicamente, vengono generate con diversi *seed*, così da ottenere statistiche più robuste e informative sui risultati conseguiti.

In secondo luogo, per mettere alla prova la capacità del modello di adattarsi a situazioni locali di traffico differenti, è stata indotta un'asimmetria direzionale, designando una riga orizzontale della griglia come arteria principale (Est-Ovest), dunque con densità di traffico maggiore delle strade trasversali, mantenendo comunque invariato il volume totale di veicoli. Tale arteria viene posta su una riga diversa per ogni variante di addestramento, al fine di scongiurare l'ipotesi che le policy sviluppate siano dipendenti dalla posizione dell'incrocio, mentre viene mantenuta fissa sulla stessa riga nelle configurazioni di validazione e di test, in modo da garantire la comparabilità dei risultati.

Training, validation e test. La Tabella 3.4 raccoglie in un unico prospetto tutte le combinazioni di griglia, spaziatura e profilo di traffico introdotte in questo lavoro, con il relativo utilizzo nella pipeline sperimentale; le varianti seed di validazione e test sono omesse, poiché non introducono combinazioni distinte da quelle elencate. In tutto possiamo contare 11 configurazioni: 3 di training, 1 di validation e 7 di test.

3.2.4 Assunzioni e limiti del modello di simulazione

Si esplicitano di seguito le semplificazioni strutturali adottate in fase di simulazione, finalizzate a ridurre la complessità computazionale e a rendere il problema trattabile:

- **Fasi transitorie:** Come precedentemente illustrato, l'intervallo semaforico di giallo è assimilato cinematicamente a un prolungamento del verde. La prevenzione dei conflitti è affidata unicamente alla fase di "tutto-rosso".

Griglia	Spaziatura	Profilo di traffico	Utilizzo	Configurazione
4×4	100m	Complete (arteria riga 0)	Train, Test	config_4x4_100m_train1
4×4	100m	Complete (arteria riga 1)	Train, Test	config_4x4_100m_train2
4×4	100m	Complete (arteria riga 2)	Train, Test	config_4x4_100m_train3
4×4	100m	Flat	Validation	config_4x4_100m_6k_flat
4×4	100m	Peak	Test	config_4x4_100m_6k_peak
4×4	200m	Flat	Test	config_4x4_200m_6k_flat
4×4	200m	Peak	Test	config_4x4_200m_6k_peak
5×5	100m	Flat	Test	config_5x5_100m_9.4k_flat
5×5	100m	Peak	Test	config_5x5_100m_9.4k_peak
6×6	100m	Flat	Test	config_6x6_100m_11.5k_flat
6×6	100m	Peak	Test	config_6x6_100m_11.5k_peak

Tabella 3.4: Tutte le combinazioni di griglia, spaziatura e profilo di traffico impiegate in questo lavoro, a meno delle varianti seed di validazione e test, con il relativo utilizzo e il nome del file di configurazione corrispondente.

- **Omogeneità topologica:** L'analisi è confinata a topologie a griglia regolare, con incroci ortogonali e segmenti stradali di lunghezza uniforme. Tale astrazione permette di isolare l'impatto della sola scala dimensionale sulle prestazioni di generalizzazione, rimuovendo le distorsioni causate dalle irregolarità geometriche tipiche delle reti urbane reali.
- **Flusso unimodale e omogeneo:** La simulazione modella esclusivamente il traffico veicolare privato. Non sono previsti pedoni, ciclisti, trasporti pubblici o mezzi di soccorso. Inoltre, il flusso è perfettamente omogeneo: ogni veicolo condivide i medesimi parametri cinematici (dimensione, accelerazione massima, velocità massima).
- **Limiti di velocità uniformi:** Il limite di velocità $v_{urb} = 11.1$ m/s è una costante globale, identica per ogni corsia della rete e per ogni veicolo generato, dunque sono assenti gerarchie stradali diverse.
- **Omogeneità degli incroci:** Ogni intersezione è vincolata a una geometria a quattro vie provvista di 8 fasi semaforiche, escludendo di fatto incroci con un numero diverso di vie.
- **Assenza di *lane-changing*:** Le corsie in approccio sono dedicate in modo esclusivo ai singoli movimenti (svolta a sinistra, dritto, svolta a destra). I veicoli vengono generati e si immettono automaticamente nella corsia compatibile con il loro percorso e il cambio di corsia (*lane-changing*) è una dinamica che non è stata implementata.

- **Viabilità bidirezionale:** L'infrastruttura di rete è composta esclusivamente da strade a doppio senso di marcia. La modellazione di strade a senso unico, per quanto frequente nei contesti urbani, non rientra nell'insieme delle configurazioni generate.

3.3 Baseline classiche

In questo capitolo vengono introdotte le due baseline classiche di riferimento nella letteratura del Traffic Signal Control: Fixed-Time e Max-Pressure. Nello specifico, pur fondandosi su paradigmi di controllo opposti, l'uno statico e l'altro adattivo, entrambi gli approcci sono accomunati dal fatto di non richiedere alcuna fase di addestramento. Proprio questa loro natura li rende i candidati ideali per essere confrontati con i modelli di Deep Reinforcement Learning.

Fixed-Time. Questo algoritmo cicla le fasi semaforiche di un incrocio in ordine fisso, cambiando fase a ogni step di decisione. Fixed-Time non osserva mai lo stato della rete, non ha alcun parametro da configurare oltre all'ordine del ciclo e non richiede training. È un approccio estremamente semplice da implementare ed è la spina dorsale della stragrande maggioranza dei sistemi semaforici a livello globale, dove le uniche modifiche proposte riguardano l'ordine delle fasi semaforiche (in cui una fase può essere ripetuta più di una volta per ciclo), la durata di ciascuna fase e una complessiva estensione o restrizione del ciclo semaforico in base all'orario giornaliero. Poiché non richiede sensori stradali in tempo reale né potenza computazionale per calcolare le decisioni, esso risulta essere estremamente economico in termine di strumentazione. Gli svantaggi principali riguardano la sua rigidità e la sua intrinseca incapacità di adattarsi alle fluttuazioni improvvise del traffico, delle caratteristiche che possono risultare fatali per il traffico in caso di eventi imprevisti.

Max-Pressure. L'algoritmo Max-Pressure proposto da (Varaiya, 2013) rappresenta un sistema di controllo reattivo che garantisce matematicamente il massimo throughput (capacità di smaltimento dei veicoli) della rete stradale senza alcuna necessità di training, partendo dalla teorica assunzione che le strade abbiano capacità infinita. Questo straordinario risultato, pur non potendo essere traslato integralmente in scenari reali dove non è rara la possibilità di una completa saturazione di una via, fa affidamento solamente a dati raccolti ad ogni istante decisionale nelle corsie adiacenti all'incrocio e a un principio di base: massimizzare la pressione di ciascuna fase semaforica. Nel dettaglio questo principio prevede che, dato un incrocio i e

una fase semaforica ϕ , la pressione della fase sia definita come la somma, su tutti i movimenti (coppia corsia di ingresso/corsia di uscita) abilitati da ϕ , della differenza tra il numero di veicoli in attesa (i.e. non in movimento) e il numero di veicoli in uscita, contati sull'intera corsia; formalmente:

$$P(i, \phi) = \sum_{(\ell_{\text{in}}, \ell_{\text{out}}) \in \text{movimenti}(\phi)} \left(n(\ell_{\text{in}}) - n(\ell_{\text{out}}) \right) \quad (3.4)$$

dove $n(\ell)$ indica il numero di veicoli nella corsia ℓ , ℓ_{in} una corsia in ingresso e ℓ_{out} una corsia in uscita, $\text{movimenti}(\phi)$ i movimenti abilitati dalla fase ϕ . Max-Pressure, a ogni step di decisione, calcola la pressione di tutte le fasi e seleziona quella a pressione massima, ovvero:

$$\phi^*(i) = \underset{\phi}{\operatorname{argmax}} P(i, \phi).$$

Uno svantaggio di questo algoritmo è dato dalla richiesta di sensoristica per monitorare le corsie per la loro interezza; inoltre non è previsto alcun vincolo sulle azioni consecutive, ciò implica che in linea di principio la stessa fase può essere selezionata per un numero indefinito di volte. Ma il problema principale di questo approccio consiste nell'assenza di garanzie di equità: infatti, in presenza di un forte afflusso di traffico su un certo insieme di corsie, la pressione maggiore potrebbe trovarsi sempre sulle fasi che coinvolgono tali vie, rendendo impossibile alle altre fasi la possibilità di essere selezionate, con conseguenti tempi di attesa eccessivi per i veicoli che si trovano sulle vie caratterizzate da un minore afflusso veicolare.

3.4 Architettura MetaSTGAT

Questo capitolo introduce il modello principale attorno a cui si sviluppa l'intero lavoro, sulla base dell'architettura originale proposta da (Wang et al., 2022): MetaSTGAT. La scelta di replicare e modificare tale modello è stata dettata dalla volontà di creare un sistema che potesse avere le caratteristiche ritenute necessarie per proporre una soluzione accettabile al problema del Traffic Signal Control secondo dei termini specifici di questo lavoro di tesi: ottenere un modello in grado di coordinare in modo efficace gli incroci di una rete stradale, così da minimizzare i tempi di percorrenza medi dei veicoli e garantire, per quanto possibile, un certo grado di equità tra i vari flussi di veicoli. A differenza di Fixed-Time, MetaSTGAT è un modello sensibile allo stato della rete per quanto riguarda la scelta delle fasi, e, diversamente da Max-Pressure, cerca un compromesso tra *throughput* (la capacità di

smaltimento dei veicoli) e *starvation* (l'eccessivo tempo di attesa dei veicoli); tutto questo a costo di una richiesta di informazioni più dettagliata da raccogliere ad ogni istante decisionale (§3.1) e a costo di una fase di addestramento.

L'architettura di questo modello di reinforcement learning si articola nei seguenti componenti, organizzati in una pipeline che elabora lo stato di ogni intersezione (s_i^t) portandolo ad una rappresentazione compatta, utilizzata per produrre i Q-value (ovvero una stima della ricompensa attesa per ciascuna azione):

- un encoder di stato duale
- una cella ricorrente (LSTM)
- due istanze di un layer di *graph attention* (GAT)
- due moduli di meta-learning, spaziale e temporale
- una layer lineare per calcolare i Q-value.

Analizzeremo qui ciascun componente citato, soffermandoci sulla ragione che ha condotto all'introduzione di tali elementi e sulle modifiche apportate all'architettura originale. Prima di introdurre la generalizzazione "meta", ci concentreremo sulla presentazione dell'architettura STGAT, ovvero la versione standard del modello priva del meccanismo di meta-apprendimento. La Figura 3.4 anticipa in forma schematica la pipeline appena elencata, utile come riferimento visivo durante la lettura dei paragrafi che seguono.

3.4.1 State Encoder duale

Lo stato $s_i^t \in \mathbb{R}^{d_0}$ ($d_0 = 32$ o 20 , §3.1) di un incrocio i al tempo t attraversa due Multilayer Perceptron (MLP) a due strati con la stessa architettura ma pesi e inizializzazioni indipendenti:

$$e_i^t = \text{MLP}_t(s_i^t) = \sigma(W_{21} \sigma(W_{11}s_i^t + b_{11}) + b_{21}) \in \mathbb{R}^{d_1} \quad (3.5)$$

$$e_j^t = \text{MLP}_s(s_i^t) = \sigma(W_{22} \sigma(W_{12}s_j^t + b_{12}) + b_{22}) \in \mathbb{R}^{d_1} \quad (3.6)$$

dove d_1 viene fissato a 64, $W_{11}, W_{12} \in \mathbb{R}^{d_0 \times d_1}$ e $W_{21}, W_{22} \in \mathbb{R}^{d_1 \times d_1}$ sono i pesi, $b_{11}, b_{21}, b_{12}, b_{22} \in \mathbb{R}^{d_1}$ i bias e σ la funzione di attivazione ReLU.

Il vettore $e_i^t = \text{MLP}_t(s_i^t)$ verrà usato come input nella LSTM e veicolerà le caratteristiche temporali dello stato, mentre $e_j^t = \text{MLP}_s(s_i^t)$ verrà usato come input nelle GAT, catturando le caratteristiche spaziali dello stesso.

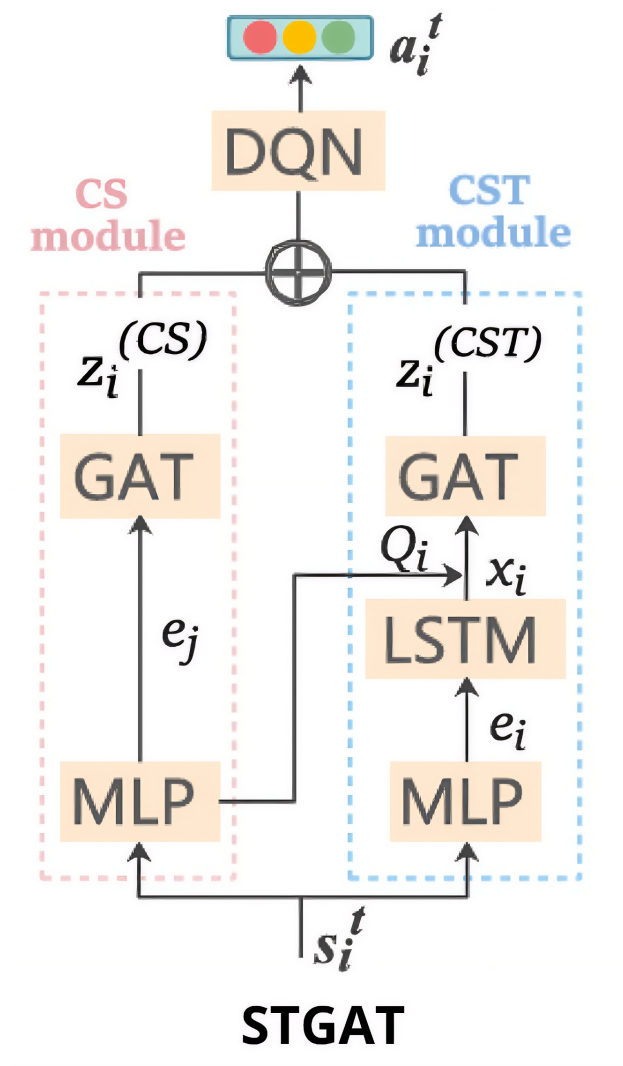


Figura 3.4: Schema a blocchi dell'architettura STGAT: dallo stato dell'intersezione s_i^t , attraverso l'encoder duale (MLP) e i moduli CS e CST (GAT e LSTM a pesi condivisi), fino all'azione a_i^t scelta dal DQN. Adattato da Wang et al. (2022).

3.4.2 Modulo spazio-temporale corrente (CST)

Nel momento di una decisione, un conteggio istantaneo dei veicoli, da solo, non distingue una coda che si sta formando da una che si sta esaurendo. Due situazioni che possono richiedere una fase diversa appaiono dunque identiche a un modello privo di memoria. Il primo dei due moduli di STGAT viene introdotto per questo motivo: combina l'informazione temporale accumulata da un'intersezione con l'informazione spaziale corrente di sé stessa e dei suoi vicini, così che la scelta dell'azione possa tenere conto di come il traffico più ravvicinato si sta evolvendo, non solo di come si presenta localmente nell'istante attuale. Altri lavori in materia cercano di applicare lo stesso concetto proponendo delle Spatio-Temporal Graph Neural Networks (STGNN) con una struttura simile a quella che introdurremo qui sotto, ma in questi modelli le informazioni spaziali e temporali sono semplicemente combinate in un unico embedding, oppure usate separatamente. Con il presente modulo si cerca di ovviare a questa limitazione intrecciando le informazioni spaziali e temporali in modo più elaborato.

Il ramo temporale dell'encoder e_i^t (§3.4.1) attraversa anzitutto una cella LSTM con pesi condivisi da ogni nodo della rete:

$$\begin{bmatrix} f_i^t \\ \iota_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(W_{\{f,\iota,o,g\}}^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f,\iota,o,g\}} \right), \quad (3.7)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \iota_i^t \odot g_i^t,$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t);$$

in notazione compatta:

$$(h_i^t, c_i^t) = \text{LSTM}(e_i^t; h_i^{t-1}, c_i^{t-1}). \quad (3.8)$$

Il vettore $x_i^t := h_i^t$ prodotto da questa cella rappresenta concettualmente un riassunto temporale dei flussi di traffico locali.

Lo stesso vettore, nel meccanismo di attenzione che segue, diventa l'informazione che i vicini dell'incrocio i "leggono" per decidere quanto pesare il contributo di i nel proprio contesto locale. Questa trasmissione di informazione avviene tramite l'applicazione di una Graph Attention Network (GAT) con una rappresentazione differente rispetto a quella della GAT standard di [Veličković et al. \(2018\)](#) riportata nel Capitolo 2. In tale GAT infatti ciascun nodo viene comparato con gli altri

nodì secondo una stessa proiezione dell’embedding. Nella struttura proposta invece, prendendo ispirazione da approcci di machine translation (Vaswani et al. (2017)), i nodi si influenzano vicendevolmente secondo valori distinti: Query, Key e Value. In particolare si scelgono come Query $Q = e_j^t$ (la codifica spaziale corrente del nodo ricevente) e come Key e Value $K = V = x_i^t$ (la memoria temporale di ciascun vicino). L’idea alla base di questa scelta è il tentativo di arricchire le caratteristiche temporali correnti di ciascun nodo con la memoria temporale dei flussi dello stesso e dei nodi vicini. Si precisa che nel caso di $H > 1$ teste di attenzione, come in questo lavoro, i valori Q, K, V non sono proiettati da matrici distinte per ciascuna testa, bensì il vettore a d_1 componenti viene partizionato in H segmenti contigui di dimensione $d_h = d_1/H$, e la testa h opera sul solo blocco $Q_h(i) := Q(i)[hd_h : (h+1)d_h]$ (analogamente per $K_h(u), V_h(u)$). Le teste restano quindi distinte perché leggono porzioni diverse dello stesso embedding, non perché dispongano di proiezioni apprese proprie.

Formalmente, nel caso in esame, per ciascun nodo i con vicinato $\mathcal{N}(i)$ (con $i \in \mathcal{N}(i)$) e h una delle $H = 4$ teste di dimensione $d_h = d_1/H = 16$, questo modulo di GAT viene descritto come segue:

$$\begin{aligned}\varphi_h(i, u) &= \frac{Q_h(i) \cdot K_h(u)}{\sqrt{d_h}}, \\ \alpha_h(i, u) &= \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))}, \\ z_h(i) &= \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u).\end{aligned}\tag{3.9}$$

L’output finale di questo modulo spazio-temporale corrente sarà

$$z_{\text{CST}}(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1}.$$

Auspiciabilmente, in questo stato è racchiusa una sintesi delle dinamiche spaziali e temporali del traffico locale e degli incroci adiacenti.

3.4.3 Modulo spaziale corrente (CS)

Il modulo spazio-temporale, filtrato dalla memoria della cella ricorrente, può attenuare segnali che sono invece rilevanti nell’istante presente; ad esempio un picco di traffico simultaneo su un intero asse necessita di coordinamento spaziale puro, a prescindere dalla durata del picco stesso. Questo secondo modulo, strutturalmente simile al primo, elabora perciò la sola informazione spaziale corrente e_j^t , senza

alcuna componente di memoria. Il vettore e_j^t attraversa una GAT che opera secondo le stesse dinamiche descritte nel paragrafo precedente, ma con l'assegnazione $Q = K = V = e_j^t$. L'output finale del modulo spaziale corrente sarà

$$z_{CS}(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1},$$

Tale rappresentazione del traffico permetterà al modello di focalizzarsi sulla coordinazione dei nodi rispetto alla sola fotografia presente della rete.

3.4.4 Testa Q-value

Le due uscite, $z_{CST}(i)$ e $z_{CS}(i)$, catturano quindi aspetti complementari, l'uno informato dalla memoria temporale dei vicini, l'altro dalla sola situazione presente. Tali vettori vengono infine concatenati e attraversano un singolo layer lineare per produrre la stima del valore di ciascuna delle 8 azioni eseguibili all'incrocio i e all'istante t :

$$q(s_i^t, \cdot) = W_q [z_{CST}(i) \parallel z_{CS}(i)] + b_q \in \mathbb{R}^8 \quad (3.10)$$

dove $W_q \in \mathbb{R}^{8 \times 2d_1}$ e $b_q \in \mathbb{R}^8$.

3.4.5 Meta-knowledge learner

I due moduli appena descritti, alla base della struttura STGAT, condividono gli stessi pesi per ogni intersezione della rete, indipendentemente dal ruolo che occupa. Ad esempio un incrocio periferico, uno pienamente interno alla rete stradale, o uno attraversato da un'arteria a densità di traffico elevata (§3.2.3) applicano esattamente la stessa regola di coordinamento. MetaSTGAT introduce un meccanismo di meta-apprendimento che condiziona tale regola su un embedding di caratteristiche locali dell'incrocio, cosicché i layer di GAT e di LSTM possano specializzarsi rispetto alle condizioni osservate da ciascun nodo invece di applicare la stessa funzione ovunque. L'idea alla base di questo meccanismo, condivisa da entrambi i componenti "meta" che seguono, è illustrata in Figura 3.5: le meta-feature spaziali e temporali, elaborate da un meta-learner dedicato, generano i pesi dei layer di LSTM e GAT al posto di parametri fissi condivisi da ogni nodo.

In particolare, due MLP a due strati, uno per l'aspetto spaziale e uno per quello temporale del problema, apprendono un embedding di caratteristiche locali dell'intersezione (Spatial Meta-Knowledge, SMK; Temporal Meta-Knowledge, TMK) a

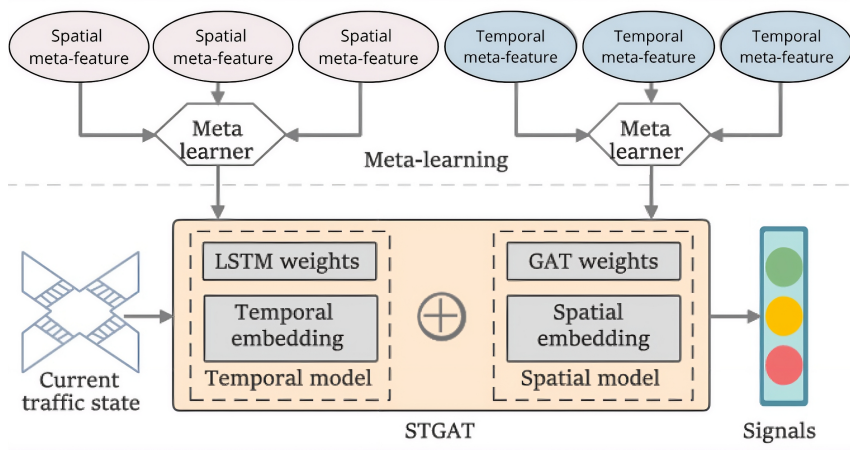


Figura 3.5: Principio del meta-apprendimento applicato da MetaSTGAT: le meta-feature spaziali e temporali condizionano, tramite un meta-learner dedicato, i pesi dei layer LSTM e GAT usati per elaborare lo stato corrente del traffico. Adattato da Wang et al. (2022).

partire dalle meta-feature che descrivono il nodo (x_i^{smk} e x_i^{tmk}):

$$\text{SMK}(i) = \text{MLP}_{\text{smk}}(x_i^{\text{smk}}) \in \mathbb{R}^{64} \quad (3.11)$$

$$\text{TMK}(i) = \text{MLP}_{\text{tmk}}(x_i^{\text{tmk}}) \in \mathbb{R}^{64} \quad (3.12)$$

Come già accennato, tali valori verranno poi usati per generare i pesi di Meta-GAT e Meta-LSTM, tramite dei MLP addizionali specializzati per ciascuno dei moduli, secondo quanto riporteremo in (§3.4.6) e (§3.4.7). Il passaggio dalle meta-feature grezze ai due embedding $\text{SMK}(i)$ e $\text{TMK}(i)$ è riassunto graficamente in Figura 3.6.

Le meta-feature x_i^{smk} e x_i^{tmk} sono pensate per fornire al modello informazioni aggiuntive che ne consentano una migliore specializzazione rispetto al contesto specifico di ciascun incrocio.

Il vettore spaziale $x_i^{\text{smk}} \in \mathbb{R}^{26}$ concatena cinque gruppi di feature, ciascuno variabile da nodo a nodo:

- pressione per corsia [12]: per ciascuna corsia, la differenza tra i veicoli in ingresso sulla corsia e la media dei veicoli sulle corsie in uscita raggiungibili da essa, divisa per 30;
- numero di incroci adiacenti [1]: frazione tra il numero di vicini del nodo e il massimo previsto dalla topologia a griglia (4), distingue quindi un incrocio d'angolo, di bordo o interno;
- differenza tra la pressione media propria e quella di ciascun incrocio adiacente [4]: per ciascuno dei vicini, la pressione media delle 12 corsie del nodo

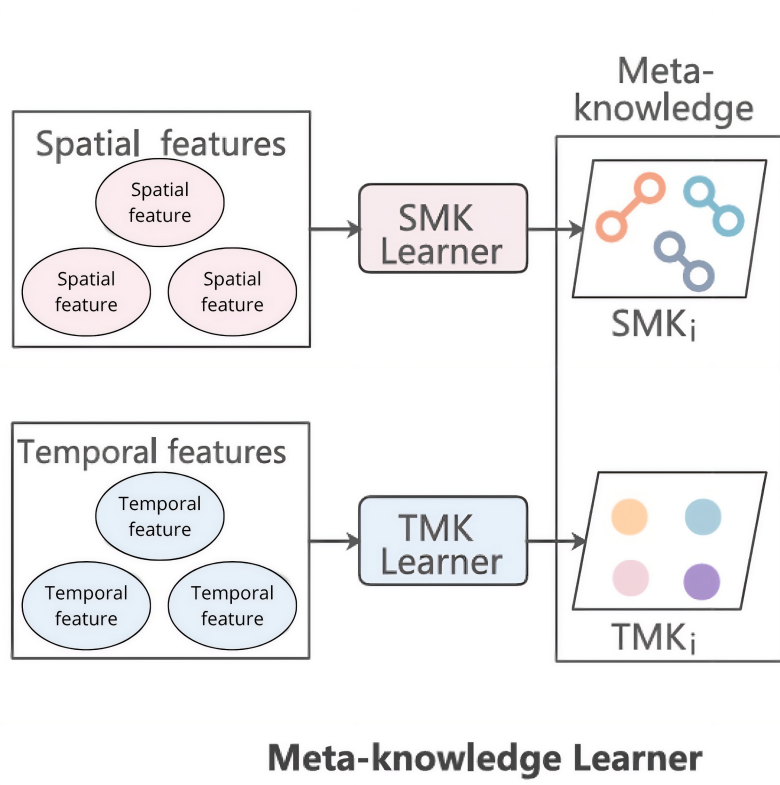


Figura 3.6: Il Meta-knowledge Learner: le meta-feature spaziali e temporali grezze vengono elaborate rispettivamente dallo SMK-Learner e dal TMK-Learner, producendo gli embedding $SMK(i)$ e $TMK(i)$ usati da Meta-GAT e Meta-LSTM. Adattato da Wang et al. (2022).

meno quella del vicino, con zero-padding negli slot oltre il numero di vicini effettivamente presenti;

- asimmetria di pressione tra le corsie N/S e le corsie E/O dello stesso nodo [1]: media della pressione sulle 6 corsie N/S meno la media sulle 6 corsie E/O;
- pressione per fase [8]: la stessa quantità usata da MaxPressure (§3.3) per decidere la fase da selezionare, una componente per ciascuna delle 8 fasi.

Il costo di raccolta di queste feature si limita a sensori con la stessa capacità visiva già richiesta per lo stato principale e per il reward, sia sulle corsie in ingresso sia su quelle in uscita: le feature di pressione si misurano semplicemente contando i veicoli su queste corsie, mentre il numero di incroci adiacenti è un dato topologico costante per ciascun incrocio.

Il vettore di meta-feature temporali $x_i^{\text{tmk}} \in \mathbb{R}^{25}$ concatena invece tre diversi tipi di informazione, tutte variabili nel tempo all'interno dello stesso nodo:

- variazione della coda per corsia rispetto a 3 passi precedenti [12]: per ciascuna corsia, differenza tra il numero di veicoli in coda all'istante corrente e quello di 3 step prima;
- durata di permanenza della fase corrente [1]: numero di step consecutivi in cui la fase è rimasta attiva, saturato e normalizzato in $[0, 1]$ rispetto a un tetto di 2 step, oltre il quale la maschera anti-starvation impone comunque un cambio di fase;
- deviazione standard della coda per corsia sulla finestra recente [12]: calcolata sugli ultimi 5 step disponibili.

Anche queste feature vengono calcolate a partire da informazioni disponibili nel dataset e non richiedono strumenti aggiuntivi.

Si noti come nessuna di queste feature entri anche nello stato principale: la scelta è legata alla volontà di non appesantire lo stato del sistema, sfruttando le stesse per agire direttamente sui pesi che regolano la trasmissione dell'informazione.

3.4.6 Meta-GAT

La versione meta delle GAT usate in STGAT riprende la struttura architetturale delle stesse, ma implementa delle modifiche mirate a consentire ai layer di apprendere regole di aggregazione spaziale dipendenti dalle caratteristiche del singolo nodo. I valori assegnati a Q, K, V rimangono invariati, sia per il modulo spaziale ($Q =$

$K = V = e_j^t$) sia per il modulo spazio-temporale ($Q = e_j^t$, $K = V = x_i^t$). L'unica differenza riguarda l'inserimento, nel punteggio di compatibilità dell'Equazione 3.9, di una trasformazione lineare per nodo che in STGAT non esiste: il modulo CS riceve il proprio meta-embedding da $\text{SMK}(i)$, il modulo CST da $\text{TMK}(i)$, in modo che la regola di attenzione spaziale e quella spazio-temporale si adattino a caratteristiche locali diverse. In particolare, per ciascun nodo i e ciascuna testa h :

$$W_{j,h}^{(i)}, b_{j,h}^{(i)} = \text{MLP}_{\text{GAT,smk},h}^{(i)}(\text{SMK}(i)) \in \mathbb{R}^{d_h \times d_h}, \mathbb{R}$$

per il modulo spaziale, e

$$W_{k,h}^{(i)}, b_{k,h}^{(i)} = \text{MLP}_{\text{GAT,tmk},h}^{(i)}(\text{TMK}(i)) \in \mathbb{R}^{d_h \times d_h}, \mathbb{R}$$

per il modulo spazio-temporale, dove le iper-reti MLP sono composte da due layer. E dunque, per ogni nodo $u \in \mathcal{N}(i)$:

$$\varphi_h(i, u) = \frac{(W_h^{(i)} Q_h(i)) \cdot K_h(u) + b_h^{(i)}}{\sqrt{d_h}}, \quad (3.13)$$

$$\alpha_h(i, u) = \frac{\exp(\varphi_h(i, u))}{\sum_{u' \in \mathcal{N}(i)} \exp(\varphi_h(i, u'))},$$

$$z_h(i) = \sum_{u \in \mathcal{N}(i)} \alpha_h(i, u) V_h(u),$$

$$z(i) = [z_1(i) \parallel \dots \parallel z_H(i)] \in \mathbb{R}^{d_1},$$

con $W_h^{(i)}$ e $b_h^{(i)}$ relativo al proprio modulo di riferimento. La Figura 3.7 mette a confronto i due moduli appena descritti, evidenziando come SMK e TMK alimentino iper-reti distinte per generare i pesi (W, b) specifici di ciascuno.

3.4.7 Meta-LSTM

Meta-LSTM applica alla cella ricorrente di STGAT la stessa generalizzazione che Meta-GAT applica al *graph attention*: i pesi $W_{(f,\iota,o,g)}$ e i bias $b_{(f,\iota,o,g)}$ condivisi in STGAT da ogni nodo della rete, sono qui sostituiti da pesi e bias generati individualmente per ogni nodo.

Un MLP a due layer per ciascun gate genera, dal solo $\text{TMK}(i)$, i pesi

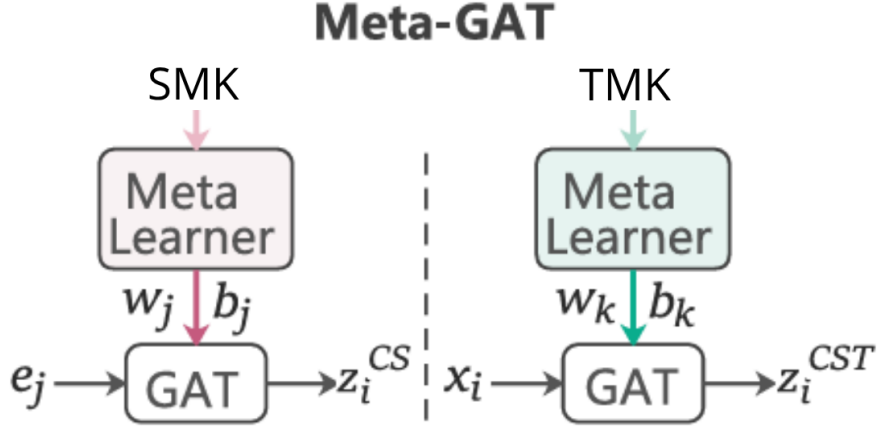


Figura 3.7: Meta-GAT: il modulo CS (sinistra) genera i pesi (W_j, b_j) della propria GAT a partire da SMK, il modulo CST (destra) genera (W_k, b_k) a partire da TMK; entrambi producono l'output z_i del rispettivo modulo. Adattato da Wang et al. (2022).

$W_{(f,\iota,o,g)}^{(i)} \in \mathbb{R}^{2d_1 \times d_1}$ e i bias $b_{(f,\iota,o,g)}^{(i)} \in \mathbb{R}^{d_1}$, modificando l'Equazione 3.7 come segue:

$$\begin{bmatrix} f_i^t \\ \iota_i^t \\ o_i^t \\ g_i^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(\left(W_{\{f,\iota,o,g\}}^{(i)} \right)^\top [e_i^t \parallel h_i^{t-1}] + b_{\{f,\iota,o,g\}}^{(i)} \right) \quad (3.14)$$

$$c_i^t = f_i^t \odot c_i^{t-1} + \iota_i^t \odot g_i^t \quad (3.15)$$

$$x_i^t = h_i^t = o_i^t \odot \tanh(c_i^t) \quad (3.16)$$

In notazione compatta, esplicitando la sola dipendenza aggiuntiva dai pesi generati dal meta-learner rispetto:

$$(h_i^t, c_i^t) = \text{LSTM}(e_i^t; h_i^{t-1}, c_i^{t-1} \mid W_{\{f,\iota,o,g\}}^{(i)}, b_{\{f,\iota,o,g\}}^{(i)}), \quad x_i^t := h_i^t. \quad (3.17)$$

L'output x_i^t alimenta il modulo CST di Meta-GAT come K e V (§3.4.6), analogamente a quanto visto con STGAT. L'architettura appena descritta è riassunta graficamente in Figura 3.8, dove W_Φ, b_Φ indicano genericamente i pesi e i bias generati dal meta-learner.

Messi insieme, i moduli descritti in questa sezione compongono l'architettura di Figura 3.9, che ne riassume il flusso complessivo dallo stato s_i^t all'azione scelta.

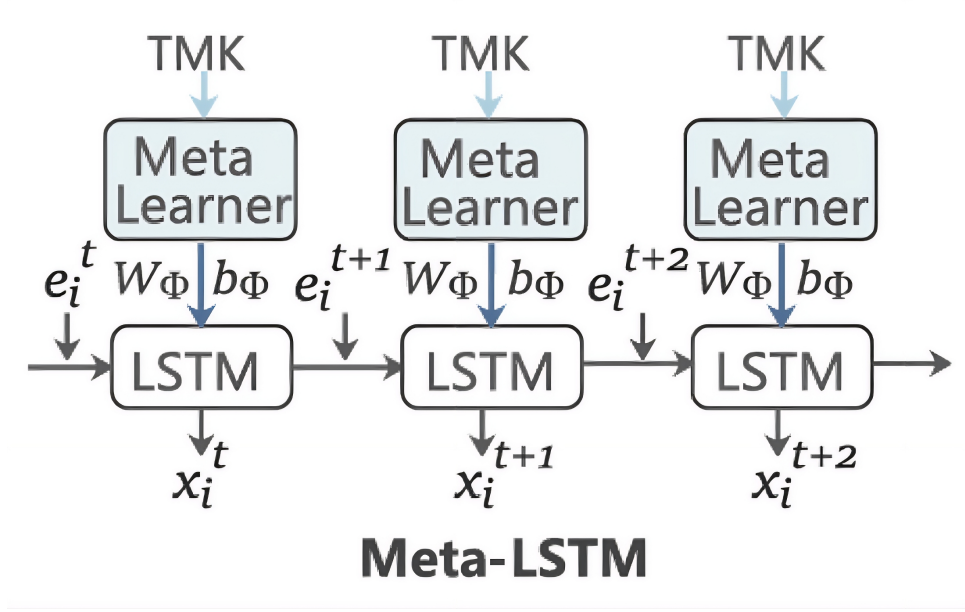


Figura 3.8: Meta-LSTM "srotolata" nel tempo: a ogni istante t il meta-learner genera, da $\text{TMK}(i)$, i pesi W_Φ e b_Φ della cella LSTM che produce x_i^t dall'input e_i^t e dallo stato nascosto precedente h_i^{t-1}, c_i^{t-1} . Adattato da Wang et al. (2022).

3.5 Differenze rispetto al modello di riferimento

Il modello proposto in questo lavoro condivide con il riferimento originale Wang et al. (2022) l'intera architettura sopra descritta. Per cercare di migliorare le prestazioni del sistema rispetto agli obiettivi di tesi, sono state introdotte alcune modifiche, elencate di seguito e argomentate singolarmente. L'efficacia di alcune di queste modifiche verrà analizzata più dettagliatamente nel Capitolo 4.

- **Limitazione del campo visivo:** un contributo originale di questo studio consiste nell'introduzione di un campo visivo limitato per ciascun incrocio, per cui i dati raccolti da ogni intersezione sono circoscritti entro un raggio di visibilità d_{vis} . Le motivazioni alla base di questa implementazione sono da ricercare in due ragioni distinte: una di carattere pratico e una di carattere teorico. La prima mira a ridurre il gap tra simulazione e realtà, per il quale è essenziale limitare l'uso della strumentazione tecnica necessaria per la raccolta dei dati ad un'area il più circoscritta possibile. La seconda trova fondamento nella cinematica dei veicoli: ogni veicolo in questa simulazione raggiunge una velocità di percorrenza massima $v_{\text{max}} = 11.1 \text{ m/s}$ (circa 40 km/h) dettata dal limite di velocità imposto su ciascuna strada. Ciò implica che in un intervallo di tempo fissato $\Delta t = \Delta t_{\text{green}} + \Delta t_{\text{yellow}} = 13 \text{ s}$, un veicolo possa percorrere

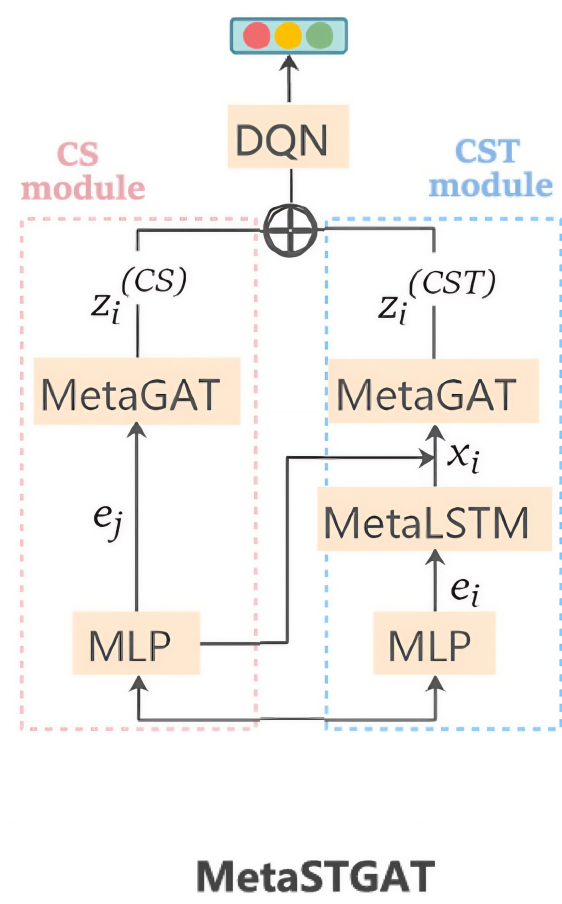


Figura 3.9: Schema a blocchi completo di MetaSTGAT: rispetto a STGAT (Figura 3.4), GAT e LSTM sono sostituiti dalle rispettive versioni meta-condizionate, con i pesi generati dagli embedding $SMK(i)$ e $TMK(i)$ del Meta-knowledge Learner. Adattato da Wang et al. (2022).

una distanza non superiore a

$$d_{\text{vis}} = v_{\text{max}} \cdot \Delta t \approx 144.3 \text{ m.}$$

Il calcolo esclude i secondi di "tutto-rosso", in cui il deflusso è interrotto. Il campo visivo è stato quindi ristretto anche per forzare l'agente RL a concentrarsi esclusivamente sui veicoli che una fase semaforica può effettivamente influenzare. Sebbene in questa architettura il cutoff spaziale sia assunto omogeneo per l'intera rete, il codice che lo implementa tiene nativamente in considerazione eventuali casi di limiti di velocità o intervalli temporali variabili. Inoltre la riduzione del campo visivo non viene applicata se la strada risulta essere più breve del cutoff stesso. Questa limitazione percettiva è comunque riservata ai soli modelli di reinforcement learning di questo studio e non viene applicata alla baseline (come MaxPressure).

- **Funzione di ricompensa:** l'articolo originale impiega esclusivamente una ricompensa di pura pressione:

$$r_i^t = -\frac{P_i^t}{100}, \quad P_i^t = \text{Incoming}_i^t - \text{Outgoing}_i^t \quad (3.18)$$

dove P_i^t è la *pressione dell'intersezione* i al tempo t , ovvero la differenza tra il numero di veicoli in entrata (Incoming_i^t) e in uscita (Outgoing_i^t) dall'intersezione al tempo t . Il modello del presente studio sostituisce questa ricompensa con (Equazione 3.3):

$$r_i^t = \text{clip} \left(\frac{-(\text{Incoming}_i^t - \text{Passed}_i^t) - \alpha \cdot \text{MaxRedWait}_i^t - \text{Penalty}_i^t}{100}, -2, 0 \right).$$

Il termine Penalty_i^t scatta quando la fase scelta non fa passare nessun veicolo pur essendocene in coda, il caso del "verde a vuoto": senza una penalizzazione esplicita l'agente può restare a lungo, nelle prime fasi di training, su una scelta degenera di questo tipo senza ricevere un segnale di correzione chiaro; questo valore aiuta quindi la convergenza. Il termine $\alpha \cdot \text{MaxRedWait}_i^t$ regola invece il concetto cardine di questa tesi: l'iperparametro α modula quanto la ricompensa penalizzi l'attesa massima registrata a un'intersezione, permettendo di studiare il compromesso tra la riduzione del tempo di percorrenza medio e il mantenimento dell'equità tra le direzioni di marcia (Capitolo 4). Infine il *clip* tra -2 e 0 , con il limite inferiore calibrato sperimentalmente nel corso della sperimentazione, evita che i picchi occasionali dei termini prece-

denti (in particolare MaxRedWait_i^t , potenzialmente illimitato) destabilizzino l'apprendimento con aggiornamenti di gradiente sproporzionati.

- **Stato dell'incrocio:** il lavoro di riferimento adotta uno stato composto solo dal numero di veicoli per ciascuna corsia d'entrata n_i^t e dalla fase semaforica corrente in codifica one-hot p_i^t (per una dimensione totale $d_0 = 20$). Si aggiunge in questo studio la componente w_i^t , il vettore dei tempi di attesa dei veicoli da più tempo in ciascuna corsia, portando d_0 a 32. La motivazione è la stessa del termine di ricompensa MaxRedWait_i^t : dare al modello un segnale diretto e locale sull'attesa, necessario perché possa correlare le proprie decisioni con l'equità tra le direzioni e non solo con il numero di veicoli in coda.
- **Maschera sulle azioni:** l'articolo di riferimento lascia all'agente la scelta libera tra tutte le 8 fasi a ogni passo. Si introduce qui il vincolo anti-starvation dell'Equazione 3.2 (§3.1), che invalida la fase corrente se già selezionata due volte consecutive. Il numero due non è arbitrario: nelle primissime fasi di training, quando la politica è ancora vicina al caso e non ha alcuna preferenza appresa, un agente senza questo vincolo tende a bloccarsi sulla prima fase che produce un ritorno anche solo leggermente positivo, ripetendola indefinitamente; vietare la ripetizione oltre la seconda volta consecutiva rompe questo ciclo senza impedire a una fase di restare attiva per un tempo ragionevole quando è davvero la scelta migliore per due passi di fila.
- **Algoritmo di Deep Reinforcement Learning:** Wang et al. (2022) impiega DQN standard. Il modello presentato adotta invece Double DQN (van Hasselt et al., 2016) (§2.1), che disaccoppia selezione e valutazione dell'azione nel calcolo del target, riducendo il bias di sovrastima del valore Q tipico di DQN. L'aggiornamento della target network è inoltre un *soft update* (Polyak averaging, $\tau = 0.01$) invece di una copia periodica integrale dei pesi: una media mobile esponenziale rende il bersaglio dell'ottimizzazione più stabile passo dopo passo, evitando i salti bruschi che una sostituzione integrale periodica introdurrebbe.
- **Calcolo del gradiente e aggiornamento dei pesi:** lo studio originale allena la componente ricorrente (LSTM) senza una fase di burn-in, calcolando il gradiente a partire dall'istanza campionata con lo stato ricorrente (h, c) non ricostruito. Qui si campionano invece sequenze di 4 transizioni dal replay buffer, con un burn-in di 2 passi secondo lo schema R2D2 (Kapturowski et al., 2019): i passi di burn-in aggiornano (h, c) senza contribuire al gradiente, correggendo

il disallineamento tra lo stato ricorrente osservato durante la raccolta dati e quello ricostruito in training, e poi i successivi 2 passi servono ad aggiornare i pesi del modello. Si applica infine un *gradient clipping*, limitando a norma 1 il gradiente, al fine di stabilizzare la convergenza.

- **Replay buffer:** il riferimento originale utilizza un replay buffer con una capacità di 10.000 transizioni e con campionamento uniforme di singole transizioni. Si adotta qui invece un replay buffer organizzato per sequenze, con Prioritized Experience Replay (PER, Schaul et al., 2016), la relativa correzione tramite pesi di *importance sampling* (§2.1) e una capacità di 4.800 transizioni. Il PER concentra l'aggiornamento dei pesi sulle sequenze con errore di stima più alto, anziché trattare queste ultime allo stesso modo delle transizioni già ben predette; limitare la capacità del buffer agli ultimi episodi, invece di conservarne l'intera storia, mantiene l'esperienza di training coerente con una politica che ha già imparato qualcosa, invece di continuare a ricampionare le traiettorie quasi casuali dei primissimi episodi.
- **Funzione di loss:** il paper di riferimento impiega l'errore quadratico medio (MSE) tra Q predetto e target. Si predilige qui la Huber loss (§2.1), più robusta agli outlier della MSE, pesata dai coefficienti di importance sampling introdotti dal PER.

3.6 Meccanismi spaziali alternativi

Il modulo Meta-GAT decide quanto ciascun vicino debba contribuire all'aggiornamento di un'intersezione tramite un punteggio di compatibilità appreso, specifico per ogni nodo (§3.4.6).

Questa formulazione rappresenta tuttavia solo una delle possibili declinazioni del paradigma di *message passing* su grafo. Con l'obiettivo di esplorare soluzioni alternative, in questo capitolo vengono introdotte ed estese in chiave meta-learning due differenti architetture di aggregazione spaziale: Meta-GCN e Meta-SONAR.

Ogni altro componente dell'architettura (encoder di stato, meta-learner spaziale e temporale, Meta-LSTM, testa Q-value) e la procedura di training (§3.7) rimarranno invariati, cosicché ogni differenza di prestazioni che verranno misurate nel Capitolo 4 sarà attribuibile al solo meccanismo di aggregazione spaziale.

La prima architettura, una convoluzione grafica (GCN), rimuove il punteggio di compatibilità della GAT e lo sostituisce con un peso fisso determinato solo dalla topologia. Il secondo, una propagazione ondulatoria a lungo raggio chiamata SO-

NAR (Trenta et al., 2025), sostituisce l'aggregazione a un passo con l'evoluzione discretizzata di un sistema dinamico su più iterazioni interne.

In entrambi i casi il principio "meta" resta identico a Meta-GAT: i pesi dell'aggregazione sono generati da un embedding SMK/TMK per ciascun nodo, non appresi come parametro fisso condiviso.

3.6.1 Meta-GCN

La GCN (Kipf and Welling, 2017), introdotta nel Capitolo 2 (Equazione 2.6), pesa il contributo di ogni vicino con la normalizzazione simmetrica del grado: una quantità che dipende solo dalla topologia della rete, mai dal contenuto dello stato dei nodi coinvolti. È dunque il termine di paragone più diretto per isolare cosa aggiunge il punteggio di compatibilità di Meta-GAT: sostituendolo con questa regola fissa, e lasciando invariato tutto il resto dell'architettura, ogni differenza di prestazioni misurata nel Capitolo 4 è imputabile a quel solo meccanismo.

Per un nodo i , con $\mathcal{N}(i)$ il suo vicinato (incluso il self-loop permanente di questo lavoro, §3.1) e $\hat{d}_i = |\mathcal{N}(i)|$ il grado corrispondente, la regola originale della GCN si scrive, per ogni vicino $u \in \mathcal{N}(i)$:

$$h'_i = \sigma \left(\sum_{u \in \mathcal{N}(i)} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W V(u) \right)$$

dove $W \in \mathbb{R}^{d_1 \times d_1}$ è una matrice di pesi condivisa da ogni nodo della rete, σ la funzione di attivazione ReLU e $V(u)$ il valore del nodo u (h_u nell'Equazione 2.6 del Capitolo 2).

Il principio meta genera W e b per ciascun nodo ricevente i , a partire dal suo embedding meta m_i (pari a $\text{SMK}(i)$ per il modulo CS e a $\text{TMK}(i)$ per il modulo CST), tramite un'iper-rete a due strati, in modo analogo a quanto visto in Meta-GAT:

$$W^{(i)}, b^{(i)} = \text{MLP}_{\text{GCN},m}^{(i)}(m_i) \in \mathbb{R}^{d_1 \times d_1}, \mathbb{R}$$

Poi, per ogni nodo i :

$$z(i) = \sigma \left(\left(\sum_{u \in \mathcal{N}(i)} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W^{(i)} V(u) \right) + b^{(i)} \right) \quad (3.19)$$

Come per Meta-GAT, il modulo CST assegna $V = x_i^t$ (l'uscita della Meta-LSTM, §3.4.7) e $m_i = \text{TMK}(i)$; il modulo CS assegna $V = e_j^t$ e $m_i = \text{SMK}(i)$.

3.6.2 Meta-SONAR

Sia Meta-GAT sia Meta-GCN propagano informazione a un solo passo di distanza per ogni applicazione del layer (Capitolo 2): raggiungere un vicino a k archi di distanza richiede k applicazioni. SONAR (Trenta et al., 2025) evita questo costo modellando la propagazione come un sistema dinamico continuo: la sua evoluzione nel tempo, non nello spazio dei layer, porta l'informazione a più hop di distanza in un'unica applicazione.

Il sistema si comporta come una rete di masse collegate da molle e smorzatori, una massa per intersezione. Raccolto lo stato di tutte le I intersezioni al tempo continuo t in $X(t) \in \mathbb{R}^{I \times d_1}$, la sua evoluzione è governata da un'equazione del second'ordine, un'accelerazione e non una semplice velocità:

$$\ddot{X}(t) = -\mathcal{L}^a X(t) W - D(X(t)) \odot \dot{X}(t) + F(X(t)) \quad (3.20)$$

Tre termini, tre ruoli distinti:

- $\mathcal{L}^a$ è un Laplaciano di grafo pesato da una conduttanza $a_{ui} \geq 0$ per arco: tira lo stato di un nodo verso quello dei suoi vicini, come una molla;
- $D(\cdot) \geq 0$ è un termine dissipativo che agisce sulla velocità $\dot{X}(t)$: frena il movimento, come un attrito;
- $F(\cdot)$ è una forzante esterna: inietta energia dall'esterno del sistema.

W è, nella formulazione originale, una matrice condivisa che mescola le d_1 componenti di ciascun nodo prima di applicare il Laplaciano; in questo lavoro W è l'identità, e il termine si riduce al solo accoppiamento spaziale pesato da a_{ui} .

Riscritta come sistema del prim'ordine, con velocità ausiliaria $V = \dot{X}$, e discretizzata con passo h su L iterazioni, l'aggiornamento per ogni iterazione è:

$$\begin{aligned} (\mathcal{L}^a X)_i &= \sum_{u \in \mathcal{N}(i)} a_{ui} (X_i - X_u), \\ V &\leftarrow V - h(\mathcal{L}^a X + D(X) \odot V - F(X)), \\ X &\leftarrow X + h \tanh(V) \end{aligned} \quad (3.21)$$

dove $\mathcal{N}(i)$ è il vicinato di i (incluso il self-loop, §3.1) e il $\tanh$ sull'aggiornamento di X è un accorgimento di stabilità numerica, assente nella derivazione originale ma presente nell'implementazione ufficiale di Trenta et al. (2025).

La formulazione originale non ha alcuna nozione di meta-apprendimento: a_{ui} , $D(\cdot)$ e $F(\cdot)$ vi sono reti a pesi fissi, condivise da tutti i nodi. Questo lavoro applica il

principio meta alla sola resistenza a_{ui} , l'analogo diretto del peso appreso per coppia di nodi di Meta-GAT e Meta-GCN: una rete a due strati genera, dall'embedding meta m_i del nodo ricevente i , i parametri $W_{\text{res}}^{(i)} \in \mathbb{R}^{d_1 \times 1}$ e $b_{\text{res}}^{(i)} \in \mathbb{R}$, da cui

$$a_{ui} = \text{ReLU}\left((X_u - X_i)W_{\text{res}}^{(i)} + b_{\text{res}}^{(i)}\right) \quad (3.22)$$

il ReLU garantisce $a_{ui} \geq 0$, coerente con l'interpretazione di a_{ui} come conduttanza. Dissipazione $D(\cdot)$ e forzante $F(\cdot)$ restano reti a pesi fissi applicate allo stato corrente del nodo: sono filtri locali, non meccanismi di comunicazione tra nodi, per cui il legame con le caratteristiche locali codificate da SMK/TMK è più debole che per la conduttanza.

La posizione iniziale X^0 e la velocità iniziale V^0 sono prodotte in modo analogo a Meta-GAT: per il modulo CST $X^0 = x_i^t$ (l'uscita della Meta-LSTM) e $V^0 = W_V e_j^t$ con $m_i = \text{TMK}(i)$, per il modulo CS: $X^0 = e_j^t$ e $V^0 = W_V e_j^t$ con $m_i = \text{SMK}(i)$.

A differenza di Meta-GAT e Meta-GCN, dove la profondità spaziale si ottiene impilando più applicazioni del layer a pesi indipendenti, in Meta-SONAR le L iterazioni dell'Equazione 3.21 condividono gli stessi pesi $W_{\text{res}}^{(i)}$: la profondità ha quindi un costo di parametri costante rispetto a L , un vantaggio strutturale dichiarato di questo meccanismo rispetto a uno stack di layer indipendenti (Trenta et al., 2025).

3.7 Procedura di training e differenze dal paper originale

I modelli proposti, a meno di ablazioni specifiche, sono addestrati con Double DQN su sequenze campionate da un replay buffer prioritizzato, con burn-in e Backpropagation Through Time (BPTT), Huber loss pesata dagli *importance-sampling weight* di PER, soft update della target network e gradient clipping (§3.5); l'ottimizzatore è RMSprop per fedeltà al metodo del paper originale.

Funzione di loss. Le componenti di Double DQN, Huber loss e PER sono definite in astratto, per una singola transizione, in §2.1; invece qui formalizzeremo il modo in cui si combinano sulle sequenze ricorrenti e multi-nodo di questo lavoro.

Sia I il numero di intersezioni della rete e ℓ_{seq} la lunghezza di una sequenza campionata k , di cui le prime istanze servono solo al burn-in per inizializzare lo stato ricorrente (h, c) , mentre le successive $\ell_{\text{seq}} -$ istanze contribuiscono al gradiente.

Per ciascun nodo i e ciascun passo t successivo al burn-in, il target Double DQN è:

$$y_i^t = r_i^t + \gamma Q(s_i^{t+1}, \operatorname{argmax}_{a'} Q(s_i^{t+1}, a'; \theta); \theta^-) \quad (3.23)$$

dove online e target network elaborano lo stesso stato ricorrente propagato dal burn-in. L'errore di differenza temporale è

$$\delta_i^t = Q(s_i^t, a_i^t; \theta) - y_i^t,$$

e la loss della sequenza k è la Huber loss media su tutti i nodi e i passi validi:

$$\mathcal{L}_k = \frac{1}{I} \sum_{i=1}^I \sum_t L_\delta(\delta_i^t) \quad (3.24)$$

Il campionamento dal replay buffer avviene per sequenze intere, non per transizioni isolate, di conseguenza anche la priorità del PER è definita a livello di sequenza, come la media di $|\delta_i^t|$ sugli stessi nodi e passi di $\mathcal{L}_k$, e aggiornata dopo ogni batch. La loss ottimizzata a ogni passo di gradiente è la media, sulle B sequenze del batch, delle loss $\mathcal{L}_k$ pesate dai rispettivi coefficienti di importance sampling w_k :

$$\mathcal{L} = \frac{1}{B} \sum_{k=1}^B w_k \mathcal{L}_k. \quad (3.25)$$

Il calcolo di $\mathcal{L}$ è seguito da un solo passo di RMSprop, dal gradient clipping e dal soft update della target network (§3.5).

Gli iperparametri usati nel presente studio sono i seguenti:

- learning rate: 10^{-3}
- γ : 0.85
- d_1 : 64
- numero di teste H : 4
- ε : da 0.9 a 0.01 con decadimento esponenziale
- batch size B : 20
- PER buffer size N : 4.800 istanze
- lunghezza sequenze ℓ_{seq} : 4 con burn-in 2

Ogni addestramento di questo lavoro ha una durata di 100 episodi.

Ciclo di training. Prima dell'inizio del training vero e proprio, 10 episodi di *warm-up* raccolgono transizioni con $\varepsilon = 1$ (azioni interamente casuali) con lo scopo di riempire il replay buffer un numero sufficiente di istanze su cui poter addestrare l'agente. Nel training vero e proprio, alla fine di ogni episodio, l'agente esegue 100 aggiornamenti del gradiente, numero ereditato dal paper originale. Il valore di ε decade una volta per episodio verso il suo minimo, raggiungendolo al 60% del numero totale di episodi pianificati per l'addestramento. Ricordiamo che usare ε -greedy come strategia di esplorazione significa che, a ogni passo di decisione, l'agente sceglie un'azione casuale con probabilità ε e altrimenti l'azione a valore Q massimo secondo la stima corrente. Oltre a questo decadimento continuo, si applica la così detta *esplorazione ciclica*, per la quale un episodio ogni 10 viene eseguito interamente a $\varepsilon = 1$ anche dopo che il valore minimo di ε è stato raggiunto, senza però contribuire all'aggiornamento dei pesi in quello specifico episodio. L'idea alla base dell'esplorazione ciclica è quella di continuare a inserire nel buffer esperienza che la politica ormai quasi deterministica non visiterebbe più spontaneamente, al fine di dare al modello la possibilità di non convergere su delle soluzioni sub-ottimali.

Capitolo 4

Confronto

Questo capitolo descrive il protocollo sperimentale di questa tesi e ne riporta i risultati. Definisce anzitutto le metriche di valutazione (§4.1) e il protocollo sperimentale (§4.2); presenta poi il confronto di base tra le baseline classiche e MetaSTGAT Pro insieme ai tre studi sperimentali di questa tesi, sul peso α della componente di equità, sull'ablation della procedura di addestramento e sui meccanismi di comunicazione spaziale alternativi (§4.3), e ne riporta i risultati (§4.4).

4.1 Metriche di valutazione

Nella letteratura sul Traffic Signal Control le metriche di valutazione proposte sono numerose. Questo capitolo introduce le 3 metriche principali utilizzate, coerentemente con il duplice obiettivo di questo lavoro, ovvero efficienza ed equità (come motivato nel Capitolo 1): il *travel time* (tempo di viaggio) e il *trip completion* (veicoli che completano il percorso) per l'efficienza, il *max wait* (l'attesa massima registrata da un veicolo ad un incrocio) per l'equità. C'è da sottolineare che né *travel time* né *trip completion* sono metriche che appaiono esplicitamente nella ricompensa dei modelli. Pertanto qualunque risultato su di esse sarà l'indiretta conseguenza di come la reward ha indirizzato l'apprendimento della policy.

Travel time. Per ogni veicolo v , il tempo di viaggio è

$$tt_v = \begin{cases} t_{\text{arrivo}}(v) - t_{\text{spawn}}(v) & \text{se } v \text{ è arrivato entro la fine dell'episodio} \\ t_{\text{fine episodio}} - t_{\text{spawn}}(v) & \text{altrimenti (un lower bound del vero valore)} \end{cases} \quad (4.1)$$

Una delle due metriche cardini in questo studio è il travel time medio $\overline{tt}$, ovvero la media di tt_v calcolata sull'intera popolazione di veicoli generati nell'episodio. Tale

metrica non considera soltanto i veicoli arrivati, ma include anche i veicoli che non hanno completato il percorso entro la fine della simulazione. La ragione è evitare un bias di sopravvivenza: un modello che congestiona la rete e lascia arrivare solo una minoranza di veicoli su percorsi ottimizzati otterrebbe, calcolando la media solo sui veicoli arrivati, un risultato positivo ma non rappresentativo del comportamento reale della rete.

La seconda vera metrica cardine in questo lavoro di tesi è il travel time massimo $\overline{tt}_{\max}$, ovvero il massimo tra tutti i travel time dei veicoli generati, anche in questo caso indipendentemente dal completamento del percorso, per motivi analoghi a quelli descritti per il travel time medio.

Trip completion. Questa metrica misura la percentuale di veicoli che hanno completato il percorso entro la fine dell’episodio, escludendo dal totale i veicoli che in assenza di semafori e in condizioni di flusso libero non riuscirebbero comunque a completare il proprio percorso. In questo studio questa metrica viene indicata con tc . Il dettaglio di tc per ogni singolo modello e ogni singola configurazione è in Tabella 4.10, a fine capitolo.

Max wait. Questa metrica riguarda invece il massimo periodo di tempo per cui un veicolo è rimasto bloccato allo stesso incrocio. Dunque questo valore non si azzerà nel momento in cui un veicolo avanza nella corsia, ma si annulla soltanto se la vettura supera l’incrocio; ciò significa che max wait non considera solamente il lasso di tempo in cui il veicolo è rimasto fermo, ma tiene conto anche del tempo in cui lo stesso si trova in movimento all’interno della corsia senza però oltrepassare l’intersezione. Essa viene calcolata prendendo il massimo valore tra tutti i veicoli e tutte le intersezioni.

Come per il travel time, include anche gli stalli ancora in corso al termine dell’episodio. In questo studio max wait viene indicato con $\text{wait}_{\max}$.

La presente metrica non va confusa con MaxRedWait_i^t , il termine usato nella ricompensa (Capitolo 3) per penalizzare l’attesa durante il training. MaxRedWait_i^t è calcolato a ogni singolo passo di decisione, su una sola intersezione alla volta, ma soprattutto è limitata al campo visivo dell’incrocio. $\text{wait}_{\max}$ è invece calcolato una sola volta a fine episodio, come massimo su tutta la rete, e fa riferimento all’intera corsia.

4.2 Protocollo sperimentale

Ogni modello RL è addestrato sulle tre configurazioni di training presentate nella Tabella 3.4, riportata per comodità di lettura qui sotto.

Griglia	Spaziatura	Profilo di traffico	Utilizzo	Configurazione
4×4	100m	Complete (arteria riga 0)	Train, Test	<code>config_4x4_100m_train1</code>
4×4	100m	Complete (arteria riga 1)	Train, Test	<code>config_4x4_100m_train2</code>
4×4	100m	Complete (arteria riga 2)	Train, Test	<code>config_4x4_100m_train3</code>
4×4	100m	Flat	Validation	<code>config_4x4_100m_6k_flat</code>
4×4	100m	Peak	Test	<code>config_4x4_100m_6k_peak</code>
4×4	200m	Flat	Test	<code>config_4x4_200m_6k_flat</code>
4×4	200m	Peak	Test	<code>config_4x4_200m_6k_peak</code>
5×5	100m	Flat	Test	<code>config_5x5_100m_9.4k_flat</code>
5×5	100m	Peak	Test	<code>config_5x5_100m_9.4k_peak</code>
6×6	100m	Flat	Test	<code>config_6x6_100m_11.5k_flat</code>
6×6	100m	Peak	Test	<code>config_6x6_100m_11.5k_peak</code>

Seguendo un ciclo fisso, le tre configurazioni si alternano episodio per episodio. A fine training, l'ultimo checkpoint e quello che ha ottenuto il valore di loss più basso durante l'intero addestramento sono confrontati sulla configurazione di validazione `config_4x4_100m_6k_flat`, mediando $\bar{tt}$ su 5 seed diversi. Questa fase serve a evitare due problemi che si concatenano tra loro. Il primo è che in un ciclo di apprendimento online la politica non converge necessariamente in modo monotono, dunque gli ultimi episodi addestrati potrebbero allontanare la policy da un punto di minimo quantomeno locale. Il secondo è che la scelta del modello che ha registrato la loss minima non garantisce le capacità di generalizzazione del modello, in quanto una loss bassa potrebbe anche essere sintomo di una policy che ha soltanto memorizzato delle condizioni particolari delle configurazioni di training, senza però aver realmente appreso una politica in grado di generalizzare.

Il modello che risulta essere migliore in questa fase di selezione, viene poi utilizzato nella fase di test. Ogni valutazione, sia in validation sia nella batteria di test che descriveremo ora, avviene con $\varepsilon = 0$: nessuna esplorazione residua, solo la politica appresa allo stato puro. Inoltre i risultati ottenuti in ciascuna configurazione testata, sono mediati su 5 seed diversi, analogamente a quanto fatto per $\bar{tt}$ nella fase di validazione.

Misurare la capacità di generalizzazione dei modelli è uno degli obiettivi dichiarati di questo lavoro (Capitolo 1), un asse che gran parte della letteratura sul controllo semaforico appreso non copre sistematicamente, in quanto la prassi più diffusa, seguita anche dall'articolo originale di MetaSTGAT (Wang et al., 2022), è addestrare e valutare un modello sulla stessa rete stradale, senza mai trasferirlo a una topologia diversa.

Per questo motivo la batteria di test di questo lavoro include sia le 3 configurazioni di training sia 7 configurazioni aggiuntive mai viste durante l’addestramento. Le prime permettono un confronto diretto con la prassi di letteratura appena descritta, riportando un risultato ottenuto nelle stesse condizioni con cui la maggior parte degli altri lavori riporta il proprio; le seconde, assenti nella maggior parte degli studi citati, misurano invece se le prestazioni osservate si confermano anche con il cambio di topologia e di condizioni di traffico.

Ogni modello, RL o baseline, è dunque valutato sistematicamente sulle stesse 10 configurazioni: le 3 varianti di training stesse (per misurare la prestazione in-distribuzione) più 7 configurazioni mai viste in training (4x4 a 200m invece di 100m, 5×5 e 6×6 a densità sia *flat* sia *peak*), usate per misurare la generalizzazione a topologie e densità di traffico diverse.

4.3 Confronti proposti

Il confronto sperimentale di questo capitolo è organizzato in tre studi indipendenti. Ognuno di essi isola un solo asse di variazione rispetto al modello di riferimento ($\alpha = 0.08$, MetaSTGAT a un layer) a cui ci riferiremo con il termine *MetaSTGAT Pro*, lasciando invariato tutto il resto. Verranno analizzati nel seguito i tre studi proposti. Ciascun confronto conterrà al suo interno, come riferimento, le due baseline classiche, Fixed-Time e MaxPressure, e il modello *MetaSTGAT Pro*.

4.3.1 Studio del peso α

Il primo studio (§4.4) pone a confronto tre varianti del modello MetaSTGAT Pro che differiscono solo per il valore del peso α riguardante la penalità anti-starvation nella ricompensa (Eq. 3.3). In particolare vengono valutati i valori di $\alpha \in \{0.08, 0.04, 0.00\}$. È il confronto più diretto per quantificare il compromesso tra efficienza ed equità dichiarato come obiettivo di questo lavoro. Variando solo questo iperparametro della ricompensa, senza toccare stato o architettura, osserveremo quanto costa in termini di travel time medio ottenere una data riduzione del travel time massimo e del max wait.

4.3.2 Studio di ablazione

Il secondo confronto (§4.4.3) isola sistematicamente ogni blocco di differenze tra l’implementazione di questo lavoro e la procedura descritta nell’articolo originale di MetaSTGAT, tramite 6 preset che vanno dalla replica fedele del paper al modello

MetaSTGAT Pro. È un confronto necessario perché né l'uno né l'altro estremo, da soli, spiegano il divario di prestazioni osservato tra i due: isolare un sottosistema alla volta permette di attribuire quel divario a cause specifiche.

Ogni preset applica un gruppo coerente di scelte configurative, isolando un sottosistema alla volta rispetto al modello MetaSTGAT Pro (§3.5):

Preset	Reward	Cosa isola rispetto a Pro
pro	custom	–
temporal	custom	burn-in + BPTT sulla Meta-LSTM
single_dqn	custom	Double DQN
vanilla_buffer	custom	PER, Huber, soft update, grad clipping, warm-up, esplorazione ciclica
environment	paper	campo visivo, feature dello stato w_i^t , maschera fasi consecutive
paper	paper	Replica fedele di Wang et al. (2022)

Tabella 4.1: Preset di ablazione e sottosistema isolato da ciascuno. La reward **custom** è definita da Eq. (3.3), mentre la reward **paper** è definita da Eq. (3.18).

4.3.3 Studio sui meccanismi spaziali

Il terzo studio (§4.4.4) sostituisce il solo meccanismo di comunicazione spaziale tra intersezioni di MetaSTGAT Pro. Vengono testate le seguenti varianti già introdotte nel Capitolo 3.6:

- Meta-GAT (il riferimento, a un layer come nella formulazione standard e a due layer)
- Meta-GCN (a uno e due layer)
- Meta-SONAR (a uno e due layer)

È il confronto che verifica se il beneficio osservato dipenda specificamente dal meccanismo di attenzione appreso, o se una qualunque forma di condivisione spaziale dell'informazione tra nodi vicini produca risultati equivalenti.

4.4 Risultati

I tre studi del precedente capitolo (§4.3) sono presentati secondo uno schema comune: una tabella mediata sulle 3 configurazioni di training, una corrispondente sulle 7 di test, un plot overlay che le riassume visivamente, un plot della variazione percentuale delle metriche principali rispetto a MaxPressure, gli eventuali casi particolari

osservabili solo scendendo alla singola configurazione, chiusi da una discussione che include, dove il risultato non era quello atteso, una possibile spiegazione. In queste tabelle, dallo studio sul peso α in poi, sono evidenziati in grassetto i due risultati migliori di ogni colonna, Fixed-Time incluso; nelle due tabelle del confronto di base (§4.4.1), che hanno solo tre righe, resta invece in grassetto il solo valore migliore.

Fixed-Time, MaxPressure e MetaSTGAT Pro ($\alpha = 0.08$, il modello di riferimento di questo lavoro) compaiono in ciascuno dei tre studi. Dunque, per non ripetere la stessa discussione su questi tre riferimenti di base, la sottosezione seguente li confronta da soli, mentre le sottosezioni successive approfondiranno i rispettivi confronti (peso α , ablazione, meccanismi spaziali).

4.4.1 Fixed-Time, MaxPressure e Pro: il confronto di base

Modello	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)	tc (%)
Fixed-Time	339.3	1030.0	470.0	84.9
MaxPressure	161.9	1070.0	955.0	96.6
MetaSTGAT Pro ($\alpha = 0.08$)	176.7	675.0	630.0	95.7

Tabella 4.2: Travel time medio, massimo, max wait e trip completion dei tre controllori presenti in ogni studio di questo capitolo, mediati sulle 3 configurazioni di training.

Modello	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)	tc (%)
Fixed-Time	460.7	1496.1	1289.1	79.9
MaxPressure	238.3	1647.4	1626.0	88.5
MetaSTGAT Pro ($\alpha = 0.08$)	242.2	1049.6	954.0	90.1

Tabella 4.3: Come Tabella 4.2, mediata sulle 7 configurazioni di test.

Fixed-Time è il controllore peggiore su quasi ogni metrica di riferimento, con l'unica eccezione del max wait in training (470.0s, il più basso dei tre): la rotazione fissa delle fasi impone per costruzione un limite alla durata di ogni attesa, un vincolo che le configurazioni di training, meno dense, non arrivano a violare. Questo approccio però non coordina il flusso tra le intersezioni, motivo per cui questo risultato non si ripropone nella batteria di test dove il traffico è più intenso, con il max wait che peggiora notevolmente. Non osservando mai lo stato della rete, resta comunque il limite superiore di travel time che qualunque strategia appresa dovrebbe poter migliorare. MaxPressure e MetaSTGAT Pro lo battono nettamente

sulle restanti metriche, confermando che la regola reattiva alla base di MaxPressure coglie un segnale reale nello stato della rete.

Tra MaxPressure e MetaSTGAT Pro il confronto è più sfumato. In training e in testing MaxPressure resta il modello con il travel time medio più basso in assoluto (rispettivamente 161.9s e 238.3s), coerente con la sua ottimalità di portata a livello di rete; MetaSTGAT Pro paga per questo un travel time leggermente peggiore sia in training (176.7s, +9.1%) sia in test (242.2s, +1.7%). Per quanto riguarda il max wait però il rapporto si inverte nettamente: MetaSTGAT Pro registra un valore il 34% più basso di MaxPressure in training e il 41% più basso in test (Tabelle 4.2-4.3). Questi risultati introducono il compromesso al centro di questo lavoro, discusso nella sua forma completa al variare di α nella prossima sottosezione. Da notare è il fatto che rispetto alle prestazioni di MaxPressure, il modello MetaSTGAT Pro ottiene risultati migliori nel training piuttosto che nel test: quanto ottenuto conferma sia l'importanza di un controllo coordinato tra semafori quando la densità veicolare aumenta, sia le capacità di generalizzazione acquisita dal modello MetaSTGAT Pro.

Il basso max wait di Fixed-Time in training merita un chiarimento: il suo $tt_{\max}$ (1030.0s) resta più del doppio del suo stesso max wait (470.0s), mentre per MaxPressure e per MetaSTGAT Pro i due valori restano vicini (rispettivamente 1070.0s contro 955.0s, e 675.0s contro 630.0s). La causa è strutturale: la rotazione fissa delle fasi non produce mai un blocco estremo isolato, ma un'attesa moderata che si ripete, quasi identica, a ogni incrocio della rete; il $tt_{\max}$ osservato è dunque la somma di molti contributi distribuiti lungo l'intero percorso del veicolo, non il riflesso di un singolo episodio. Negli altri due controllori vale il caso opposto: un max wait elevato è quasi sempre un episodio isolato su un solo incrocio, che da solo spiega quasi per intero il $tt_{\max}$ corrispondente, con il resto del tragitto gestito in modo efficiente.

Sul completamento (tc) la separazione più netta è quella tra Fixed-Time e tutti gli altri modelli di questo capitolo: 84.9%/79.9% (training/test) contro un intervallo compreso tra il 95.0% e il 98.3% in training e tra l'88.9% e il 95.8% in test per ogni altra variante (Tabella 4.10, a fine capitolo); l'unica eccezione parziale è MetaSTGCN, il cui completamento più basso è discusso a parte in §4.4.4. Con questo unico scarto rilevante già osservato qui, le sottosezioni seguenti non ripetono la discussione di tc configurazione per configurazione: si limitano a segnalarne la tendenza generale, rimandando alla stessa tabella per il dettaglio completo.

4.4.2 Studio del peso α : risultati

Questa sottosezione confronta i due controllori di riferimento (§4.4.1) con le tre varianti del modello Pro che differiscono solo per il peso α della penalità anti-starvation,

con $\alpha \in \{0.08, 0.04, 0.00\}$.

Modello	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	339.3	1030.0	470.0
MaxPressure	161.9	1070.0	955.0
MetaSTGAT Pro ($\alpha = 0.08$)	176.7	675.0	630.0
MetaSTGAT Pro ($\alpha = 0.04$)	170.1	750.0	690.0
MetaSTGAT Pro ($\alpha = 0.00$)	178.6	925.0	885.0

Tabella 4.4: Travel time medio, massimo e max wait, mediati sulle 3 configurazioni di training.

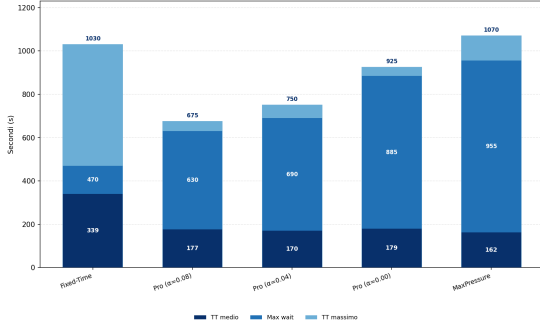
Modello	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	460.7	1496.1	1289.1
MaxPressure	238.3	1647.4	1626.0
MetaSTGAT Pro ($\alpha = 0.08$)	242.2	1049.6	954.0
MetaSTGAT Pro ($\alpha = 0.04$)	236.4	1181.6	1125.9
MetaSTGAT Pro ($\alpha = 0.00$)	248.7	1569.9	1539.4

Tabella 4.5: Come Tabella 4.4, mediata sulle 7 configurazioni di test.

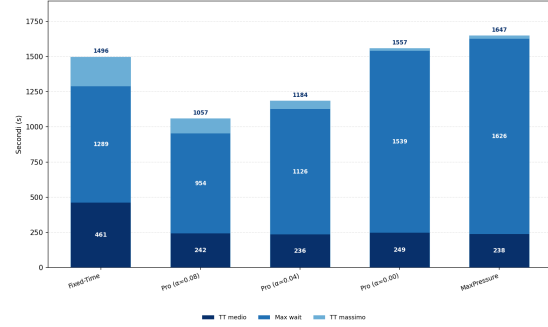
Sul travel time medio ciò che verrebbe naturale attendersi è che al crescere del valore di α la metrica in questione peggiori, con le due configurazioni prive di penalità anti-starvation, MaxPressure e $\alpha = 0.00$, che risulterebbero essere le migliori, in quanto nessuna delle due paga un compromesso con l’equità, quindi nessuna dovrebbe peggiorare travel time medio in cambio di un vantaggio che non persegue. Mentre questa analisi preliminare su α si conferma per i modelli con $\alpha = 0.08$ e $\alpha = 0.04$, $\alpha = 0.00$ risulta essere la peggiore delle tre varianti Pro su questa metrica in entrambi i regimi (178.6s in training contro 176.7s di $\alpha = 0.08$, 248.7s in test contro 242.2s). Una causa plausibile è che con $\alpha = 0$ il termine di ricompensa legato all’attesa scompare, ma il vettore w_i^t resta nello stato osservato dalla rete (§3.1); la rete continua quindi a vedere un segnale che la ricompensa non penalizza più causando un disallineamento tra stato e obiettivo che può disturbare l’addestramento.

Un ulteriore risultato da sottolineare è quello ottenuto con $\alpha = 0.04$ sul travel time medio in test, che non solo risulta essere il migliore tra le varianti del modello Pro, ma supera anche MaxPressure (236.4s contro 238.3s), confermando che una quantità di penalità anti-starvation non nulla ma sufficientemente contenuta non ostacola necessariamente la performance su travel time medio. Il risultato regge sia sulla media assoluta sia sulla media delle variazioni percentuali per singola configurazione (Figura 4.2), quindi non è l’effetto di poche configurazioni favorevoli: è un

segnale che la struttura coordina bene il traffico, con un margine che cresce proprio sulle topologie più complesse e dense.

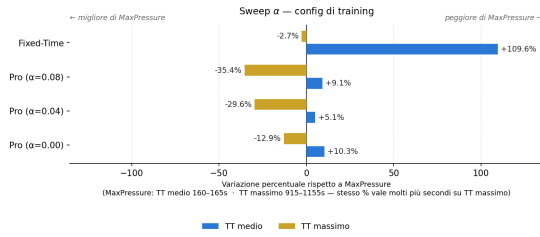


(a) Configurazioni di training

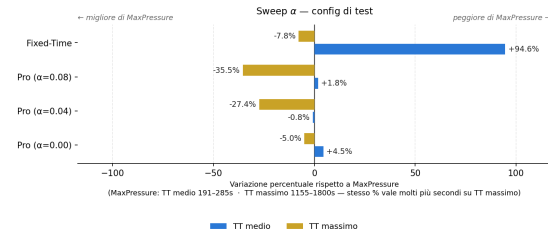


(b) Configurazioni di test

Figura 4.1: Travel time medio (fascia scura), max wait (fascia intermedia) e travel time massimo (fascia chiara) per lo sweep su α , stessi dati delle Tabelle 4.4-4.5.



(a) Configurazioni di training



(b) Configurazioni di test

Figura 4.2: Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) per lo studio su α , calcolata per singola configurazione e poi mediata.

Su max wait e $tt_{\max}$ invece il quadro è quello atteso: entrambe le metriche peggiorano in modo pressoché monotono al diminuire di α , cioè al ridursi del peso dato al caso di starvation più estremo. Il riferimento $\alpha = 0.08$ ottiene sia il $tt_{\max}$ più basso in entrambi i regimi, sia il max wait più basso, il 34% in meno di MaxPressure in training e il 41% in meno in test. In test $\alpha = 0.04$ resta comunque nettamente sotto Fixed-Time su entrambe le metriche (max wait 1125.9s e $tt_{\max}$ 1181.6s, contro 1289.1s e 1496.1s); $\alpha = 0.00$ invece pareggia o peggiora leggermente i risultati di Fixed-Time (max wait 1539.4s contro 1289.1s, $tt_{\max}$ 1569.9s contro 1496.1s).

Nonostante il vantaggio di $\alpha = 0.04$ sul travel time medio, il resto di questo lavoro adotta $\alpha = 0.08$ come riferimento standard per gli studi di ablazione e dei meccanismi spaziali (§4.4.3, §4.4.4). La ragione è la priorità dichiarata tra le metriche di questo lavoro: $\alpha = 0.04$ è sistematicamente meno incisivo di $\alpha = 0.08$ su max wait e $tt_{\max}$, le due metriche di caso peggiore che questa tesi tiene in primo

piano, a fronte di un guadagno sul solo travel time medio, la meno critica delle tre. $\alpha = 0.08$ resta quindi la scelta più coerente con l'obiettivo del lavoro, anche al costo di qualche secondo in più di travel time medio.

4.4.3 Studio di ablazione: risultati

Questa sottosezione confronta il modello Pro ($\alpha = 0.08$, riferimento) con i cinque preset di ablation presenti nella Tabella 4.1: **paper** (replica fedele del riferimento originale), **environment** (campo di visibilità, w_i^t e maschera azioni rimossi), **temporal** (burn-in e BPTT disattivati), **single_dqn** (Double DQN sostituita da DQN) e **vanilla_buffer** (PER, Huber, soft update, gradient clipping, warm-up ed esplorazione ciclica disattivati), tutti allo stesso $\alpha = 0.08$ del riferimento.

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	339.3	1030.0	470.0
MaxPressure	161.9	1070.0	955.0
Pro ($\alpha = 0.08$, riferimento)	176.7	675.0	630.0
paper	187.8	950.0	915.0
environment	179.1	1065.0	1020.0
temporal	184.8	645.0	605.0
single_dqn	174.6	595.0	485.0
vanilla_buffer	171.5	695.0	560.0

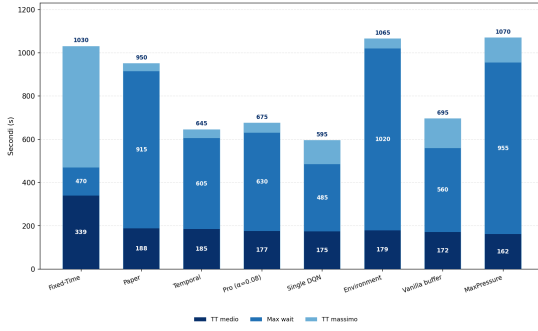
Tabella 4.6: Travel time medio, massimo e max wait sui preset di ablazione, mediati sulle 3 configurazioni di training.

Modello	$\bar{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	460.7	1496.1	1289.1
MaxPressure	238.3	1647.4	1626.0
Pro ($\alpha = 0.08$, riferimento)	242.2	1049.6	954.0
paper	271.1	1527.4	1513.3
environment	252.0	1595.6	1564.7
temporal	256.6	1169.1	1029.9
single_dqn	240.6	986.1	838.7
vanilla_buffer	240.9	1048.3	804.0

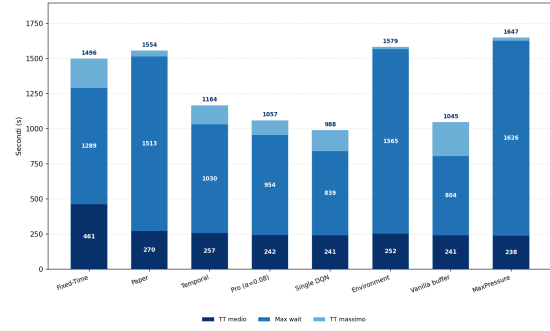
Tabella 4.7: Come Tabella 4.6, mediata sulle 7 configurazioni di test.

Sul travel time medio in training **vanilla_buffer**, **single_dqn**, il modello Pro ed **environment** restano in una fascia stretta (171.5-179.1s). La stessa quasi coincidenza tra **vanilla_buffer**, **single_dqn** e Pro si ripete in test (240.6-242.2s), ma **environment** se ne stacca visibilmente (252.0s). Il distacco non è uniforme sulle

sette configurazioni di test: sulle due a 200m è più del doppio rispetto alla media delle cinque a 100m (+7.5% contro +2.9%, calcolato per singola configurazione). La causa più diretta e più probabile è la disattivazione del cutoff di visibilità propria di questo preset. Infatti, in seguito a un addestramento comune su sole griglie a 100m, il passaggio a griglie a 200m produce con il preset in questione degli stati di valori mai osservati prima, in quanto **environment** osserva l'intera corsia e dunque registra un conteggio veicoli mai incontrato in training. Gli altri preset invece limitano questo scarto numerico grazie al ridotto campo visivo, aiutando di fatto il modello a generalizzare su reti con spaziatura maggiore rispetto a quella del training.

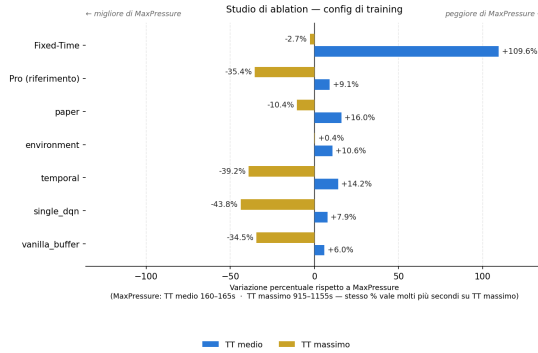


(a) Configurazioni di training

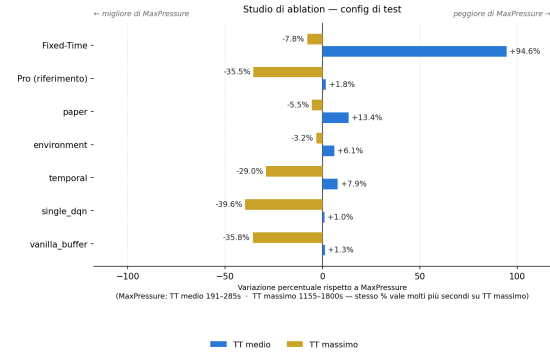


(b) Configurazioni di test

Figura 4.3: Travel time medio, max wait e travel time massimo sui preset di ablation, stessi dati delle Tabelle 4.6-4.7.



(a) Configurazioni di training



(b) Configurazioni di test

Figura 4.4: Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) sui preset di ablation, calcolata per singola configurazione/seed e poi mediata.

Su tt_{max} e $wait_{max}$, **paper** ed **environment** restano nettamente indietro in entrambi i regimi, coerentemente con l'unico elemento che condividono e che differisce da ogni altro preset: l'assenza del termine legato all'attesa nella ricompensa.

Sempre sulle stesse due metriche in questione invece, i tre preset che lasciano intatti stato, ricompensa e maschera (`temporal`, `single_dqn`, `vanilla_buffer`), registrano in training i migliori risultati in assoluto tra tutti i preset considerati, modello Pro incluso.

Tuttavia, per quanto riguarda i risultati in regime di test il quadro si inverte per `temporal`, con Pro che registra $tt_{\max}$ e $wait_{\max}$ migliori. Un’analisi sulle singole configurazioni rivela che il crollo è sistematico e cresce con la scala della griglia: il suo $tt_{\max}$ medio per configurazione resta vicino al riferimento sulle tre configurazioni 4x4 (855-975s) ma sale a 1218-1524s su 5x5 e 6x6. La causa probabilmente da ricercare nel fatto che `temporal` disattiva BPTT e burn-in: senza propagazione del gradiente lungo la sequenza né un periodo di adattamento dello stato ricorrente all’inizio di ogni episodio, il modulo temporale della Meta-LSTM ha meno modo di generalizzare il proprio stato interno a topologie di scala diversa da quella di training, mentre lo stesso stato interno resta adeguato sulla scala 4x4 già vista. La curva di training offre un indizio indipendente coerente con questa lettura: la loss di `temporal` resta stabilmente più alta di quella del riferimento per l’intero addestramento, un divario compatibile con un limite strutturale imposto dall’assenza di BPTT e burn-in.

Il vantaggio di `single_dqn` e `vanilla_buffer` su $tt_{\max}$ e max wait regge anche scendendo alla singola configurazione, senza che questo lavoro riesca a indicarne con certezza la causa. Resta aperta una domanda distinta, che uno studio dedicato dovrebbe verificare: se la generalizzazione di questi due preset regga altrettanto bene su spaziature stradali diverse da quella di training. Un indizio non conclusivo in questo senso viene dal max wait: il vantaggio di entrambi i preset sul modello Pro, ampio sulle configurazioni 5x5 e 6x6 mai viste in training, si riduce fino a scendere al di sotto del modello Pro sulle due configurazioni a 200m. Non possiamo pertanto dire con certezza che questi preset funzionino altrettanto bene su spaziature stradali diverse da quella di training.

4.4.4 Studio sui meccanismi spaziali: risultati

Questa sottosezione confronta il modello Pro (MetaSTGAT, un layer) con le sue varianti a meccanismo spaziale alternativo definite in §3.6: MetaSTGAT a due layer, MetaSTGCN a uno e due layer, e MetaSTSONAR con $L = 2$ e $L = 4$ ricorrenze. Tutti i modelli condividono l’architettura residua, gli iperparametri e procedura di training.

Il divario più netto di questo confronto separa MetaSTGCN dagli altri due meccanismi. In test, entrambe le varianti MetaSTGCN registrano valori nettamente peggiori su ogni colonna (travel time medio 288.5-333.8s contro 237.9-246.6s di Me-

Modello	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	339.3	1030.0	470.0
MaxPressure	161.9	1070.0	955.0
MetaSTGAT, 1L (Pro, riferimento)	176.7	675.0	630.0
MetaSTGAT, 2L	174.6	635.0	510.0
MetaSTGCN, 1L	248.5	1340.0	1280.0
MetaSTGCN, 2L	217.8	1180.0	1160.0
MetaSTSONAR, $L=2$	170.6	630.0	535.0
MetaSTSONAR, $L=4$	170.2	660.0	525.0

Tabella 4.8: Travel time medio, massimo e max wait sui meccanismi spaziali, mediati sulle 3 configurazioni di training.

Modello	$\overline{tt}$ (s)	$tt_{\max}$ (s)	$\text{wait}_{\max}$ (s)
Fixed-Time	460.7	1496.1	1289.1
MaxPressure	238.3	1647.4	1626.0
MetaSTGAT, 1L (Pro, riferimento)	242.2	1049.6	954.0
MetaSTGAT, 2L	246.6	1128.0	990.0
MetaSTGCN, 1L	333.8	1659.0	1631.6
MetaSTGCN, 2L	288.5	1457.6	1428.9
MetaSTSONAR, $L=2$	237.9	1038.4	862.3
MetaSTSONAR, $L=4$	238.3	1119.9	1047.4

Tabella 4.9: Come Tabella 4.8, mediata sulle 7 configurazioni di test.

taSTGAT/MetaSTSONAR), con un $tt_{\max}$ e un $wait_{\max}$ che restano vicini a quelli di MaxPressure pur trattandosi di un modello addestrato con lo stesso obiettivo di equità del riferimento. La differenza concettuale descritta in §3.6.1, un peso di aggregazione fissato dalla sola topologia contro un punteggio di compatibilità appreso per coppia di nodi, si traduce quindi in un divario di prestazioni ampio su questo compito. Un secondo layer in MetaSTGCN migliora la variante a singolo layer su tutte le metriche riportate in entrambi i regimi ($\overline{tt}$ di training 248.5→217.8s, $tt_{\max}$ di test 1659.0→1457.6s, max wait di test 1631.6→1428.9s): la profondità aggiuntiva compensa qui, almeno in parte, l'assenza di un meccanismo di compatibilità appreso, pur restando lontana dalle prestazioni di MetaSTGAT e MetaSTSONAR.

MetaSTGAT e MetaSTSONAR hanno prestazioni pressochè pari tra loro su travel time medio e massimo; sul max wait, però, è MetaSTSONAR con $L = 2$ a ottenere il risultato migliore dell'intero confronto (862.3s in test), davanti sia al modello Pro di riferimento (954.0s) sia alla sua variante a due layer (990.0s).

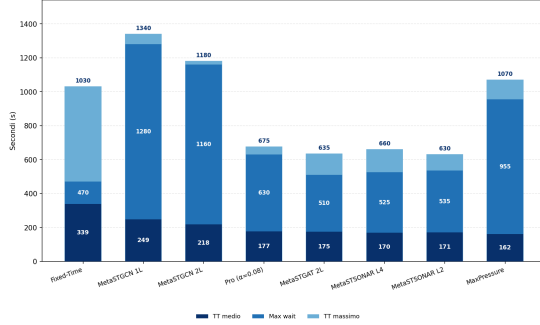
Tra le due profondità di MetaSTSONAR, $L = 2$ supera $L = 4$ anche sul $tt_{\max}$ in entrambi i regimi (630.0s contro 660.0s in training, 1038.4s contro 1119.9s in test), senza il vantaggio teorico di campo recettivo più ampio atteso da una profondità doppia.

Aggiungere un secondo layer a MetaSTGAT, a differenza di MetaSTGCN, non produce qui un miglioramento netto: la variante a due layer resta vicina al riferimento su travel time medio e massimo ma peggiora leggermente il max wait in test (990.0s contro 954.0s).

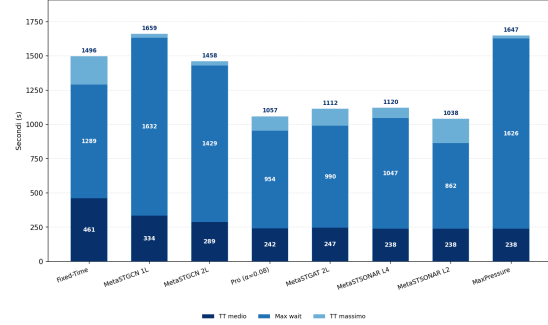
Un caso particolare nelle configurazioni di training (Tabella 4.13): MetaSTGCN a un layer ottiene un max wait medio di 1280s, peggiore di MaxPressure stesso (955s), pur condividendo con il riferimento la stessa ricompensa orientata all'equità. È il solo modello di tutto questo lavoro, tra quelli addestrati con il termine anti-starvation pieno, a non riuscire a battere la baseline reattiva su questa metrica nel proprio stesso regime di addestramento, coerentemente con la spiegazione architetturale data sopra: un peso di aggregazione fissato dalla topologia non ha modo di imparare a dare priorità al segnale del vicino più in sofferenza, per quanto la ricompensa lo incentivi a farlo.

4.4.5 Tabelle dei risultati.

Sono qui di seguito riportati i risultati completi di tutti i modelli rispetto ciascuna configurazioni testata.

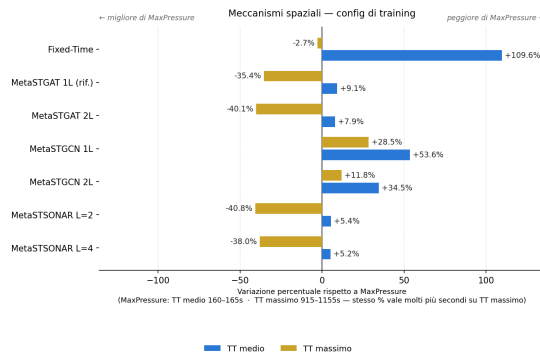


(a) Configurazioni di training

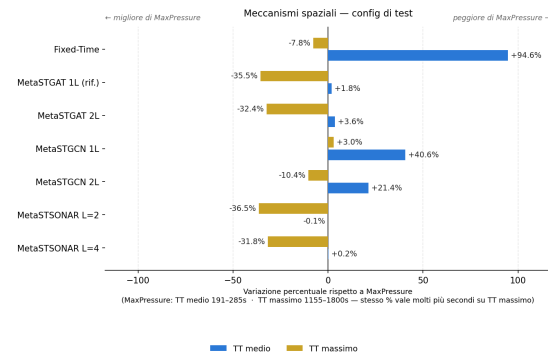


(b) Configurazioni di test

Figura 4.5: Travel time medio, max wait e travel time massimo sui meccanismi spaziali, stessi dati delle Tabelle 4.8-4.9.



(a) Configurazioni di training



(b) Configurazioni di test

Figura 4.6: Variazione percentuale di travel time medio e massimo rispetto a MaxPressure (riferimento a 0%) sui meccanismi spaziali, calcolata per singola configurazione/seed e poi mediata.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	84.6	85.5	87.9	84.7	84.0	81.9	84.4	82.6	81.5	80.4
MaxPressure	98.0	98.3	97.1	92.8	93.5	92.0	92.0	91.2	90.7	89.2
Pro ($\alpha=0.08$)	96.4	97.5	96.8	93.3	95.3	92.8	93.5	93.1	92.9	92.8
Pro ($\alpha=0.04$)	97.2	97.4	97.2	94.0	95.8	93.6	93.3	92.5	92.4	92.0
Pro ($\alpha=0.00$)	96.2	96.9	96.8	92.9	93.7	92.2	93.2	92.3	92.2	91.1
MetaSTGCN 2L	96.5	97.4	97.8	93.7	94.5	92.6	93.7	92.2	93.1	92.3
Ablation: paper	95.0	96.5	96.3	91.6	93.5	90.2	91.4	90.8	90.7	88.9
Ablation: temporal	95.8	95.6	97.0	92.7	93.4	91.6	93.2	92.5	92.8	92.6
Ablation: single_dqn	96.8	97.6	97.0	93.6	95.6	93.4	93.3	92.3	92.5	92.2
Ablation: environment	95.9	96.6	95.7	91.1	92.8	90.8	92.1	91.7	91.4	89.2
Ablation: vanilla_buffer	96.1	97.7	97.5	93.8	95.7	92.9	93.7	92.8	92.9	92.0
MetaSTGCN 1L	90.6	91.1	91.7	87.2	88.3	86.2	88.9	85.8	86.7	85.1
MetaSTGCN 2L	93.3	94.8	93.6	91.0	92.6	89.8	91.4	90.2	90.6	89.4
MetaSTSONAR L=2	97.3	97.7	97.5	93.4	95.5	93.1	93.9	92.6	93.3	92.0
MetaSTSONAR L=4	97.2	97.6	97.6	93.4	95.1	92.6	93.2	92.9	93.3	92.9

Tabella 4.10: Percentuale di trip completion tc per ogni modello e ogni configurazione, corretta escludendo dal denominatore i veicoli strutturalmente impossibilitati ad arrivare in tempo. Nessun modello scende sotto l'80%; a parte Fixed-Time (baseline non adattiva) l'unico caso ricorrente sotto il 90% è MetaSTGCN 1L.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	351.6	342.9	323.3	437.3 $\pm$ 6.4	415.5 $\pm$ 0.7	438.8 $\pm$ 4.9	459.7 $\pm$ 4.7	485.0 $\pm$ 5.1	484.8 $\pm$ 6.2	503.7 $\pm$ 4.6
MaxPressure	165.4	160.2	160.0	218.8 $\pm$ 2.3	197.3 $\pm$ 4.2	247.8 $\pm$ 2.6	217.8 $\pm$ 3.3	258.6 $\pm$ 4.9	244.7 $\pm$ 5.5	283.3 $\pm$ 1.4
Pro ($\alpha=0.08$)	180.2	176.3	173.5	230.9 $\pm$ 2.6	199.5 $\pm$ 2.4	252.0 $\pm$ 3.3	224.8 $\pm$ 3.1	266.7 $\pm$ 3.4	243.3 $\pm$ 3.4	278.6 $\pm$ 4.0
Pro ($\alpha=0.04$)	172.5	169.4	168.4	223.3 $\pm$ 3.1	196.9 $\pm$ 3.4	248.2 $\pm$ 3.8	213.4 $\pm$ 4.6	258.2 $\pm$ 3.0	237.1 $\pm$ 3.0	277.5 $\pm$ 4.5
Pro ($\alpha=0.00$)	183.5	180.8	171.5	237.5 $\pm$ 3.8	206.1 $\pm$ 3.5	257.8 $\pm$ 2.6	230.9 $\pm$ 4.5	269.8 $\pm$ 3.7	247.9 $\pm$ 4.6	290.7 $\pm$ 6.0
MetaTGAT 2L	181.3	175.0	167.6	231.8 $\pm$ 3.5	203.7 $\pm$ 2.1	252.7 $\pm$ 2.9	228.8 $\pm$ 3.7	275.4 $\pm$ 5.5	245.3 $\pm$ 4.0	289.5 $\pm$ 3.7
Ablation: paper	194.1	182.9	186.5	253.9 $\pm$ 4.6	217.7 $\pm$ 3.6	287.3 $\pm$ 4.8	245.3 $\pm$ 11.4	294.5 $\pm$ 5.2	275.8 $\pm$ 2.6	318.6 $\pm$ 3.5
Ablation: temporal	189.8	190.0	174.6	247.4 $\pm$ 6.4	213.6 $\pm$ 7.0	262.5 $\pm$ 6.8	240.3 $\pm$ 6.0	282.1 $\pm$ 4.7	256.6 $\pm$ 3.6	293.8 $\pm$ 4.1
Ablation: single_dqn	178.4	173.4	172.1	227.3 $\pm$ 1.8	198.8 $\pm$ 3.0	249.2 $\pm$ 3.8	221.8 $\pm$ 2.2	266.7 $\pm$ 5.0	241.1 $\pm$ 4.3	279.0 $\pm$ 3.5
Ablation: environment	182.3	181.7	173.1	241.0 $\pm$ 4.3	216.7 $\pm$ 7.8	268.3 $\pm$ 3.0	228.8 $\pm$ 2.4	268.7 $\pm$ 4.9	249.8 $\pm$ 8.1	292.2 $\pm$ 9.9
Ablation: vanilla_buffer	177.9	170.2	166.5	230.3 $\pm$ 2.9	202.2 $\pm$ 4.1	252.4 $\pm$ 4.2	220.1 $\pm$ 4.5	264.8 $\pm$ 3.9	238.8 $\pm$ 3.2	278.9 $\pm$ 6.4
MetaTGATCN 1L	250.5	254.6	240.5	325.6 $\pm$ 8.1	290.4 $\pm$ 6.6	343.6 $\pm$ 7.6	310.6 $\pm$ 6.3	357.8 $\pm$ 13.0	331.5 $\pm$ 9.5	376.9 $\pm$ 3.4
MetaTGATCN 2L	225.4	215.1	212.7	280.1 $\pm$ 5.0	242.7 $\pm$ 4.4	299.3 $\pm$ 3.8	266.7 $\pm$ 5.5	308.6 $\pm$ 9.4	290.7 $\pm$ 6.0	331.7 $\pm$ 2.7
MetaSTSONAR L=2	174.1	170.1	167.5	225.3 $\pm$ 5.0	196.4 $\pm$ 3.4	246.6 $\pm$ 3.1	219.5 $\pm$ 4.1	261.3 $\pm$ 4.6	235.4 $\pm$ 3.9	281.0 $\pm$ 1.8
MetaSTSONAR L=4	171.9	171.1	167.7	224.8 $\pm$ 4.4	198.3 $\pm$ 2.4	249.2 $\pm$ 3.2	221.5 $\pm$ 3.2	261.5 $\pm$ 3.7	238.5 $\pm$ 2.9	274.5 $\pm$ 2.6

Tabella 4.11: Travel time medio $t\bar{t}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	1020.0	975.0	1095.0	1410.0 $\pm$ 49.3	1431.0 $\pm$ 30.9	1290.0 $\pm$ 19.0	1620.0 $\pm$ 32.9	1482.0 $\pm$ 17.5	1653.0 $\pm$ 32.0	1587.0 $\pm$ 37.2
MaxPressure	915.0	1155.0	1140.0	1194.0 $\pm$ 20.3	1692.0 $\pm$ 133.6	1446.0 $\pm$ 105.0	1800.0 $\pm$ 0.0	1800.0 $\pm$ 0.0	1800.0 $\pm$ 0.0	1800.0 $\pm$ 0.0
Pro ($\alpha=0.08$)	780.0	600.0	645.0	888.0 $\pm$ 63.9	786.0 $\pm$ 53.3	876.0 $\pm$ 7.3	1242.0 $\pm$ 52.8	1143.0 $\pm$ 44.9	1269.0 $\pm$ 36.2	1197.0 $\pm$ 6.0
Pro ($\alpha=0.04$)	675.0	750.0	825.0	1077.0 $\pm$ 116.3	858.0 $\pm$ 84.5	1074.0 $\pm$ 79.7	1494.0 $\pm$ 275.7	1227.0 $\pm$ 24.0	1302.0 $\pm$ 105.4	1257.0 $\pm$ 51.4
Pro ($\alpha=0.00$)	885.0	1050.0	840.0	1275.0 $\pm$ 195.1	1470.0 $\pm$ 190.9	1317.0 $\pm$ 119.4	1779.0 $\pm$ 42.0	1674.0 $\pm$ 154.6	1722.0 $\pm$ 99.7	1665.0 $\pm$ 82.2
MetaSTGAT 2L	615.0	660.0	630.0	933.0 $\pm$ 54.8	714.0 $\pm$ 43.1	867.0 $\pm$ 101.5	1302.0 $\pm$ 93.6	1248.0 $\pm$ 58.8	1476.0 $\pm$ 61.2	1242.0 $\pm$ 43.9
Ablation: paper	930.0	975.0	945.0	1212.0 $\pm$ 142.7	1257.0 $\pm$ 93.1	1326.0 $\pm$ 145.3	1800.0 $\pm$ 0.0	1794.0 $\pm$ 12.0	1800.0 $\pm$ 0.0	1686.0 $\pm$ 152.0
Ablation: temporal	630.0	705.0	600.0	975.0 $\pm$ 43.5	855.0 $\pm$ 65.7	948.0 $\pm$ 47.8	1350.0 $\pm$ 93.0	1218.0 $\pm$ 36.0	1524.0 $\pm$ 47.1	1281.0 $\pm$ 75.1
Ablation: single_dqn	585.0	570.0	630.0	822.0 $\pm$ 41.8	774.0 $\pm$ 57.4	894.0 $\pm$ 70.7	1062.0 $\pm$ 131.6	1065.0 $\pm$ 75.3	1212.0 $\pm$ 39.6	1086.0 $\pm$ 63.4
Ablation: environment	1020.0	1065.0	1110.0	1341.0 $\pm$ 188.7	1446.0 $\pm$ 191.3	1416.0 $\pm$ 143.1	1746.0 $\pm$ 58.2	1689.0 $\pm$ 132.0	1743.0 $\pm$ 47.8	1671.0 $\pm$ 107.6
Ablation: vanilla_buffer	660.0	675.0	750.0	954.0 $\pm$ 38.7	795.0 $\pm$ 42.4	936.0 $\pm$ 43.1	1161.0 $\pm$ 102.4	1071.0 $\pm$ 33.7	1215.0 $\pm$ 26.8	1182.0 $\pm$ 22.0
MetaSTGCN 1L	1590.0	1335.0	1095.0	1689.0 $\pm$ 131.3	1593.0 $\pm$ 204.4	1569.0 $\pm$ 106.3	1536.0 $\pm$ 169.3	1728.0 $\pm$ 122.7	1728.0 $\pm$ 81.8	1770.0 $\pm$ 60.0
MetaSTGCN 2L	1215.0	1125.0	1200.0	1290.0 $\pm$ 89.0	1344.0 $\pm$ 244.0	1263.0 $\pm$ 99.2	1719.0 $\pm$ 102.0	1434.0 $\pm$ 134.7	1692.0 $\pm$ 114.4	1461.0 $\pm$ 121.0
MetaSTSONAR L=2	585.0	585.0	720.0	915.0 $\pm$ 79.9	735.0 $\pm$ 52.0	879.0 $\pm$ 46.1	1077.0 $\pm$ 95.1	1170.0 $\pm$ 80.5	1278.0 $\pm$ 29.1	1215.0 $\pm$ 128.0
MetaSTSONAR L=4	600.0	675.0	705.0	945.0 $\pm$ 122.6	774.0 $\pm$ 66.8	939.0 $\pm$ 122.5	1233.0 $\pm$ 150.7	1311.0 $\pm$ 97.9	1392.0 $\pm$ 58.8	1245.0 $\pm$ 51.1

Tabella 4.12: Travel time massimo $t_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi.

Modello	Train 1	Train 2	Train 3	4x4 Pk	4x4-200 F	4x4-200 Pk	5x5 F	5x5 Pk	6x6 F	6x6 Pk
Fixed-Time	465.0	285.0	660.0	984.0 $\pm$ 54.2	1053.0 $\pm$ 75.5	1014.0 $\pm$ 64.8	1572.0 $\pm$ 80.7	1314.0 $\pm$ 102.4	1620.0 $\pm$ 65.0	1467.0 $\pm$ 83.5
MaxPressure	795.0	1050.0	1020.0	1146.0 $\pm$ 29.4	1680.0 $\pm$ 138.1	1419.0 $\pm$ 98.4	1788.0 $\pm$ 24.0	1779.0 $\pm$ 42.0	1785.0 $\pm$ 30.0	1785.0 $\pm$ 30.0
Pro ($\alpha=0.08$)	735.0	540.0	615.0	798.0 $\pm$ 46.9	606.0 $\pm$ 71.4	795.0 $\pm$ 21.2	1077.0 $\pm$ 194.3	1074.0 $\pm$ 93.2	1137.0 $\pm$ 106.2	1191.0 $\pm$ 18.0
Pro ($\alpha=0.04$)	645.0	690.0	735.0	1050.0 $\pm$ 127.3	687.0 $\pm$ 99.3	1005.0 $\pm$ 94.9	1479.0 $\pm$ 281.4	1182.0 $\pm$ 99.5	1257.0 $\pm$ 88.2	1221.0 $\pm$ 43.1
Pro ($\alpha=0.00$)	855.0	990.0	810.0	1254.0 $\pm$ 204.0	1428.0 $\pm$ 229.7	1284.0 $\pm$ 122.4	1779.0 $\pm$ 42.0	1665.0 $\pm$ 165.4	1713.0 $\pm$ 100.6	1653.0 $\pm$ 75.5
MetaSTGAT 2L	495.0	555.0	480.0	876.0 $\pm$ 45.1	603.0 $\pm$ 58.8	819.0 $\pm$ 79.7	972.0 $\pm$ 183.3	1239.0 $\pm$ 49.8	1236.0 $\pm$ 221.0	1185.0 $\pm$ 26.8
Ablation: paper	885.0	945.0	915.0	1071.0 $\pm$ 109.7	1215.0 $\pm$ 91.5	1278.0 $\pm$ 135.3	1800.0 $\pm$ 0.0	1788.0 $\pm$ 24.0	1800.0 $\pm$ 0.0	1641.0 $\pm$ 171.4
Ablation: temporal	570.0	675.0	570.0	819.0 $\pm$ 112.1	789.0 $\pm$ 87.3	882.0 $\pm$ 58.0	1074.0 $\pm$ 72.6	1188.0 $\pm$ 25.8	1236.0 $\pm$ 189.6	1221.0 $\pm$ 35.0
Ablation: single_dqn	510.0	450.0	495.0	726.0 $\pm$ 30.9	606.0 $\pm$ 58.9	759.0 $\pm$ 76.8	855.0 $\pm$ 171.6	981.0 $\pm$ 124.9	948.0 $\pm$ 124.9	996.0 $\pm$ 111.7
Ablation: environment	990.0	990.0	1080.0	1335.0 $\pm$ 195.3	1422.0 $\pm$ 211.6	1401.0 $\pm$ 149.6	1734.0 $\pm$ 51.6	1671.0 $\pm$ 142.5	1728.0 $\pm$ 53.2	1662.0 $\pm$ 101.9
Ablation: vanilla_buffer	585.0	570.0	525.0	753.0 $\pm$ 85.1	594.0 $\pm$ 50.7	777.0 $\pm$ 59.5	768.0 $\pm$ 235.3	837.0 $\pm$ 17.5	834.0 $\pm$ 99.8	1065.0 $\pm$ 70.4
MetaSTGCN 1L	1485.0	1290.0	1065.0	1647.0 $\pm$ 147.7	1542.0 $\pm$ 237.6	1509.0 $\pm$ 94.2	1527.0 $\pm$ 169.5	1719.0 $\pm$ 134.0	1716.0 $\pm$ 86.2	1761.0 $\pm$ 70.7
MetaSTGCN 2L	1185.0	1125.0	1170.0	1263.0 $\pm$ 96.0	1308.0 $\pm$ 238.2	1203.0 $\pm$ 86.1	1713.0 $\pm$ 97.9	1401.0 $\pm$ 141.2	1674.0 $\pm$ 118.7	1440.0 $\pm$ 131.5
MetaSTSONAR L=2	510.0	510.0	585.0	753.0 $\pm$ 146.5	603.0 $\pm$ 32.0	720.0 $\pm$ 9.5	813.0 $\pm$ 55.6	1125.0 $\pm$ 100.8	831.0 $\pm$ 64.1	1191.0 $\pm$ 149.3
MetaSTSONAR L=4	555.0	525.0	495.0	882.0 $\pm$ 134.0	642.0 $\pm$ 52.3	816.0 $\pm$ 125.4	1137.0 $\pm$ 187.4	1263.0 $\pm$ 83.5	1368.0 $\pm$ 84.5	1224.0 $\pm$ 43.1

Tabella 4.13: Max wait $\text{wait}_{\max}$ (s) per ogni modello e ogni configurazione. Sulle configurazioni di test, media $\pm$ deviazione standard su 5 seed diversi. Sulle config 4x4-200, valori a piena corsia.

Bibliografia

- Andrea Cini, Ivan Marisca, Daniele Zambon, and Cesare Alippi. Graph deep learning for time series forecasting. *ACM Computing Surveys*, 57(12):321, 2025. doi: 10.1145/3742784.
- Zhiyuan Hu, Bowen Liu, Jianming Hu, Zihang Wang, Lihui Peng, and Yi Zhang. Mpmnlight: A self-attention enhanced message passing graph neural network for multi-intersection traffic signal control. *Electronics*, 15(8):1655, 2026. doi: 10.3390/electronics15081655.
- Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- Pablo Alvarez Lopez, Michael Behrisch, Laura Bieker-Walz, Jakob Erdmann, Yun-Pang Flötteröd, Robert Hilbrich, Leonhard Lücken, Johannes Rummel, Peter Wagner, and Evamarie Wießner. Microscopic traffic simulation using sumo. In *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018. doi: 10.1109/ITSC.2018.8569938.
- Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- Afshin Oroojlooy, Davood Hajinezhad, Mohammadreza Nazari, and Jorge Silva. Attendlight: Universal attention-based reinforcement learning model for traffic signal control. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

- Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations (ICLR)*, 2016.
- Chao Sun, Yuhao Yang, Jiacheng Li, Weiyi Fang, and Peng Zhang. A multi-agent regional traffic signal control system integrating traffic flow prediction and graph attention networks. *Systems*, 14(1):47, 2026. doi: 10.3390/systems14010047.
- Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- Alessandro Trenta, Alessio Gravina, and Davide Bacciu. Sonar: Long-range graph propagation through information waves. In *39th Conference on Neural Information Processing Systems (NeurIPS 2025)*, 2025.
- Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016.
- Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.
- Jing Wang, Xiaopeng Wang, Yang Mo, Chao Huang, Yanan Li, Guanghe Li, and Hao Zhang. Attention-based spatio-temporal graph convolutional networks-enhanced deep reinforcement learning for adaptive traffic signal control in urban traffic networks. *Scientific Reports*, 2026. doi: 10.1038/s41598-026-61371-3.
- Min Wang, Libing Wu, Man Li, Dan Wu, Xiaochuan Shi, and Chao Ma. Meta-learning based spatial-temporal graph attention network for traffic signal control. *Knowledge-Based Systems*, 250:109166, 2022. doi: 10.1016/j.knosys.2022.109166.
- Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu, and Zhenhui Li. Colight: Learning network-level

- cooperation for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, 2019. doi: 10.1145/3357384.3357902.
- Feng Xiao, Jiaming Lu, Lu Li, Wenwen Tu, and Chaojing Li. Advances in reinforcement learning for traffic signal control: a review of recent progress. *Intelligent Transportation Infrastructure*, 2025:liaf009, 2025. doi: 10.1093/iti/liaf009.
- Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haili Jin, and Zhenhui Li. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference (WWW)*, 2019. doi: 10.1145/3308558.3314139.
- Huichu Zhang, Chang Liu, Weinan Zhang, Guanjie Zheng, and Yong Yu. Generalight: Improving environment generalization of traffic signal control via meta reinforcement learning. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1783–1792, 2020. doi: 10.1145/3340531.3411859.
- Liang Zhang, Qiang Wu, Jun Shen, Linyuan Lu, Bo Du, and Jianqing Wu. Expression might be enough: representing pressure and demand for reinforcement learning based traffic signal control. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022.